

# Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

## 1 Summary: Optical Switched Networks vs. Other Computer Networks

Optical Switched Networks (OSNs) — the family that includes the three canonical designs studied here, Opera [1], Shale [2], Sirius [3], and the GA-evolved topologies introduced in §6 — move packet forwarding out of the electrical domain and into optical circuits established between Top-of-Rack (ToR) nodes. They differ from the dominant alternatives — traditional electrical packet-switched fabrics (*Clos*, *fat-tree*, *leaf-spine*), direct-connect networks (*torus*, *HyperX*, *Dragonfly*), and hybrid optical/electrical designs — on five axes that drive both the benchmarking framework (§1.2) and the architectural choices throughout this paper.

### OSN pros (advantages over electrical / hybrid networks).

- **Power and cost efficiency.** Eliminating electrical packet switches at the network core cuts power by 2–4× for equivalent bandwidth; Sirius reports 74–77% lower power than a non-blocking electrical fabric of the same size [3].
- **Line-rate independence.** Optical circuits carry any modulation format transparently, so upgrading link speed (e.g. 100 Gb/s → 400 Gb/s) does not require replacing the core switching fabric.
- **Low and predictable latency for in-circuit traffic.** When a flow traverses a direct optical circuit, there is no packet-level queuing inside the core — propagation plus a fixed guardband dominate (~100 ns for Sirius, ~200 ns for Shale). Electrical Clos fabrics add nondeterministic queuing at each hop.
- **High bisection bandwidth at scale.** Reconfigurable optical fabrics deliver full bisection to arbitrary demand matrices within a cycle (Opera), while traditional fat-trees require  $k$ -port switches that scale poorly beyond a few thousand ToRs.

- **Topology flexibility via reconfiguration.** The circuit-schedule or GA-evolved graph can be tailored to workload, whereas Clos topology is fixed at build time.

**OSN cons (disadvantages relative to traditional networks).**

- **Reconfiguration gap.** Every circuit-switched design pays a duty-cycle loss  $\delta/T$  (§1.2.5) that electrical packet switches do not incur; depending on the hardware generation this is 3.84 ns (Sirius), up to 10  $\mu$ s (Opera), or  $\approx 0$  for static photonic wiring.
- **Adversarial fragility.** As §9.1 shows, each OSN has a *routing-structural* critical load  $\rho^*$  (Sirius 0.85, Opera 0.883, Shale  $1/(h+1)$ ) above which throughput collapses. Electrical Clos fabrics have graceful degradation without a sharp cliff.
- **Bandwidth tax from multi-hop routing.** Short flows detoured through VLB intermediates (Sirius, Shale) or multi-hop expander paths (Opera) consume  $1 - 1/\ell$  of extra link time without delivering useful data (§1.2.4). Packet-switched fat-trees deliver every byte on the ingress–egress shortest path.
- **Oblivious scheduling.** Opera and Sirius use demand-*oblivious* round-robin schedules [1, 3], which means they cannot adapt to traffic imbalance within a cycle. Modern electrical fabrics run demand-aware ECMP/adaptive routing and redistribute load continuously.
- **Buffering constraints.** OSNs either eliminate buffers in the optical core (Sirius [3]) or rely on edge buffering (Opera, Shale), so bursts that exceed the configured buffer depth cause queue divergence at VLB intermediaries — the failure mode Sirius-adversarial (§9.1.3) exploits.

**How these guidelines shape the GA design.** The evolved GA-Robust topology (§6) is deliberately constructed to *adhere to the OSN pros and avoid the cons*:

- *Static photonic wiring*  $\Rightarrow$  **hw\_reconfig\_ratio** = 0: the reconfiguration-gap con vanishes (same structural advantage as Shale; better than Opera’s 4% and Sirius’s 3.84%).
- *Multi-objective fitness on {Uniform, Skewed, Hotspot}*: evolves short paths that avoid the adversarial-fragility con (§9.1.4, §9.3).
- *Target degree  $D$  tunable to ToR radix*: preserves the power and line-rate pros (the evolved graph is buildable with the same number of fibres per ToR as Opera or Sirius).
- *No forced VLB spray*: any source–destination pair routes on its *shortest* path on the evolved graph, sidestepping the bandwidth-tax con.

- *No demand-oblivious schedule*: the GA is demand-*informed* at evolve time (the fitness reads the traffic distribution), converting the oblivious-scheduling con into a design-time optimisation step.

The explicit construction of the GA fitness (§6) and the dynamic routing variant (§6.4) both inherit these guidelines; any future architecture or modification to the code base should be evaluated against this five-axis pros/cons list before being accepted.

## 2 Benchmarking Methodology

We evaluate our network architectures using a unified benchmarking framework that implements a complete feedback loop: **Benchmark Scenario** → **Experiment Configuration** → **Metrics** → **Benchmark Score**.

### 2.1 Benchmark Scenario

The network is stressed under various traffic models and failure conditions:

- **Traffic Models** (implemented in `Traffic.Benchmarks.py`): Demand matrix  $D[i, j]$  for  $i \neq j$ ;  $D[i, i] = 0$ .
  - **Uniform**:  $D[i, j] = 1$  (all pairs equal).
  - **Skewed**: Zipf/power-law over randomised node ranks. Each source  $i$  draws a uniformly random permutation  $\pi_i$  of the other  $N-1$  destinations; demand to the rank- $r$  destination is

$$D[i, j] = \frac{1}{r_{\pi_i}(j)^\gamma + 1}, \quad r_{\pi_i}(j) \in \{1, \dots, N-1\}$$

with  $\gamma$  the Zipf exponent (default 2.0). Using a random permutation per source means each seed produces a *different spatial pattern* while preserving the same marginal Zipf distribution — addressing the limitation that a fixed index-distance formula always places the same pair of nodes in the same “locality” class.

**Exact skew ratio** (max to min demand per source):

$$\frac{D_{\max}}{D_{\min}} = \frac{1/(1^\gamma + 1)}{1/((N-1)^\gamma + 1)} = \frac{(N-1)^\gamma + 1}{2} \approx \frac{(N-1)^\gamma}{2}$$

For  $N = 9$ ,  $\gamma = 2.0$ : ratio  $\approx 32 \times$  ( $D_{\max} \approx 0.50$ ,  $D_{\min} \approx 0.015$ ). For  $N = 9$ ,  $\gamma = 1.0$ : ratio  $\approx 4 \times$  ( $D_{\max} \approx 0.50$ ,  $D_{\min} \approx 0.111$ ).

- **Hotspot**:  $D[i, j] = \omega$  if  $i \in \mathcal{H}$  or  $j \in \mathcal{H}$ , else 1;  $\mathcal{H}$  hotspot nodes,  $\omega$  intensity (e.g. 10).
- **Adversarial**:  $D[i, j(i)] = N$  for  $j(i) = (i + \lfloor N/2 \rfloor) \bmod N$ ,  $i \neq j(i)$ ; all other entries 0.
- **Load Factor Sweep**: We sweep the load  $L$  from light load (0.05) to saturation (0.95).

## 2.2 Performance Metrics Definition

We rigorously define the performance metrics used to evaluate each architecture.

### 2.2.1 Normalized Throughput ( $\mathcal{T}_{norm}$ )

In the benchmarking framework (`TrafficBenchmarks.py`), all rates are expressed as a fraction of the per-link line rate  $R$ , and the implementation takes  $R = 1$  without loss of generality. Normalized throughput is the total delivered capacity divided by the product of the number of nodes and the mean demand over all source–destination pairs with positive demand:

$$\mathcal{T}_{norm} = \frac{\sum_{(i,j)} r_{\text{eff}}(i,j)}{N \cdot \bar{D}} \quad (1)$$

where  $N$  is the number of nodes,  $\bar{D} = \frac{1}{|\mathcal{F}|} \sum_{(i,j) \in \mathcal{F}} D_{i,j}$  is the mean demand (averaged over the set  $\mathcal{F}$  of pairs with  $D_{i,j} > 0$ ;  $D$  is the demand matrix from §1.1), and  $r_{\text{eff}}(i,j)$  is the effective rate delivered for pair  $(i,j)$  after the architecture-specific capacity model.

**Effective rate  $r_{\text{eff}}$ .** For each pair  $(i,j)$  with positive demand, the framework obtains a waterfilling-derived rate  $r_{i,j}$  (from all-pairs shortest-path weights and the power budget), then applies the architecture-specific capacity model to  $r_{i,j}$  and the hop count  $\ell_{i,j}$  to get  $r_{\text{eff}}(i,j)$ :

- **Shale [2]:**  $r_{\text{eff}} = r/u_{\text{max}}$ , where  $u_{\text{max}} = \max_{(a,b) \in E} \sum_{f \ni (a,b)} D_f/k_f$  is the maximum directed-link load when each flow  $f$  with demand  $D_f$  is split uniformly across  $k_f$  diverse spray paths found by penalised Dijkstra (`spray_short`). This reflects Shale’s Valiant Load Balancing (VLB) routing, where traffic is sprayed through  $h$  intermediate nodes to ensure uniform load distribution across all links [2].
- **Opera [1]:**  $r_{\text{eff}} = \alpha \cdot r \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha) \cdot r/\max(1, \ell)$  (bulk fraction  $\alpha$ , reconfig delay  $\delta$ , cycle  $T_{\text{cycle}}$ ; short flows penalized by hop count  $\ell$ ). Opera’s key insight is that the vast majority of bytes reside in bulk flows [1]; by buffering these until a direct 1-hop circuit is available, Opera eliminates the bandwidth tax on most traffic while routing the small fraction of latency-sensitive flows over a time-varying expander graph [1].
- **Sirius [3]:**  $r_{\text{eff}} = \frac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho)$ , where the hop denominator is

$$\mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \text{ (direct scheduling)} \\ 2 & \ell = 2 \text{ (Valiant detour)} \\ \ell^2 & \ell \geq 3 \text{ (multi-hop)} \end{cases}$$

scheduling efficiency  $\eta = (T_{\text{slot}} - \delta)/T_{\text{slot}} = (100 \text{ ns} - 3.84 \text{ ns})/100 \text{ ns} = 0.9616$  [3], and the load penalty is  $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$  when

$\rho > 0.85$ , else 1. Sirius achieves nanosecond-granularity reconfiguration through custom tunable laser chips with sub-912 ps tuning latency, enabling 3.84 ns end-to-end reconfiguration atop 50 Gbps channels [3].

The concrete parameter values used in the implementation (`ArchitectureParams` in `Traffic_Benchmarks.py`) are:

Table 1: Architecture-Specific Constants for  $r_{\text{eff}}$  and unified hardware reconfiguration overhead

| Arch.     | Parameter                                         | Ratio               | Physical               | Meaning                             |
|-----------|---------------------------------------------------|---------------------|------------------------|-------------------------------------|
| Shale     | $h$ (spray depth)                                 | —                   | —                      | VLB intermediate hops               |
|           | $T_{\text{slot}}$                                 | —                   | 5.632 ns               | Timeslot duration                   |
|           | $\gamma$ (hop overhead)                           | 1.2                 | 6.76 ns                | Cycle overhead per spray hop        |
|           | $\beta_0$ (base delay)                            | 2.5                 | 14.08 ns               | Base processing latency             |
|           | <b>hw_reconfig_ratio</b> $\delta/T_{\text{slot}}$ | <b>0.0000</b>       | 0 ns                   | No reconfiguration gap              |
| Opera     | $\alpha$ (bulk frac.)                             | 0.92                | —                      | Fraction of bytes on direct circuit |
|           | $\delta$ (reconfig delay)                         | $\delta/T_c = 0.04$ | $\sim 10 \mu\text{s}$  | Raw switch reconfiguration time     |
|           | $\varepsilon$ (slice)                             | —                   | 90 $\mu\text{s}$       | Topology-slice duration             |
|           | $T_{\text{cycle}}$                                | 50.0                | $\sim 250 \mu\text{s}$ | Full rotation cycle                 |
|           | <b>hw_reconfig_ratio</b> $\delta/T_c$             | <b>0.0400</b>       | 10/250 $\mu\text{s}$   | 4% of cycle lost to reconfig        |
| Sirius    | $T_{\text{slot}}$                                 | 1.0                 | 100 ns                 | Timeslot duration                   |
|           | $\delta$ (reconfig)                               | 0.0384              | 3.84 ns                | Laser tuning time                   |
|           | $\eta$ (efficiency)                               | 0.9616              | —                      | $1 - \delta/T_{\text{slot}}$        |
|           | <b>hw_reconfig_ratio</b> $\delta/T_{\text{slot}}$ | <b>0.0384</b>       | 3.84 ns/100 ns         | 3.84% of slot lost to retuning      |
| GA-Robust | (static topology)                                 | —                   | —                      | No circuit reconfiguration          |
|           | <b>hw_reconfig_ratio</b>                          | <b>0.0000</b>       | —                      | Zero overhead; static wiring        |

### 2.2.2 Flow Completion Time ( $FCT_{\text{norm}}$ )

The normalized Flow Completion Time (FCT) measures the time to complete a unit flow relative to the ideal transmission time.

$$FCT_{\text{norm}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \frac{t_{\text{end}}(f) - t_{\text{start}}(f)}{S_f/R} \quad (2)$$

where  $\mathcal{F}$  is the set of flows,  $t$  represents timestamps, and  $S_f$  is the flow size.

### 2.2.3 Mean Latency ( $\bar{\tau}$ )

Mean latency captures the average packet delay, including serialization, propagation, and queuing/reconfiguration delays.

$$\bar{\tau} = \frac{1}{N_{\text{pkts}}} \sum_p (T_{\text{prop}} + T_{\text{queue}} + T_{\text{reconfig}}) \quad (3)$$

- **Shale:**  $\tau \approx \gamma \cdot \bar{\ell}_{\text{spray}} + \beta_0$ , where  $\bar{\ell}_{\text{spray}}$  is the mean hop count across the spray paths actually used,  $\gamma = 1.2$  (6.76 ns per hop at  $T_{\text{slot}} = 5.632$  ns) is

the per-hop cycle overhead, and  $\beta_0 = 2.5$  (14.08 ns) is the base processing delay. For example, with  $h = 4$ :  $\tau \approx 1.2 \times 4 + 2.5 = 7.3$  normalized units  $\approx 41.1$  ns. Amir et al. show that Shale achieves a maximum intrinsic latency of  $O(h \sqrt[4]{N})$  timeslots and worst-case throughput of  $1/(2h)$  of line rate for  $h$  spray hops, a Pareto optimal tradeoff for ORN designs [2].

- **Opera:** At low load,  $\tau \approx \ell \cdot \varepsilon$  where  $\varepsilon = 90 \mu\text{s}$  is the topology-slice duration [1]. At higher load, bulk traffic queues for its scheduled direct circuit, adding an M/D/1 queuing component:

$$\tau(\ell, \rho) \approx \ell \cdot \delta + \min\left(\frac{\rho_{\text{bulk}} \cdot T_{\text{cycle}}}{2(1 - \rho_{\text{bulk}})}, 5 T_{\text{cycle}}\right), \quad \rho_{\text{bulk}} = \alpha \cdot \frac{\rho}{\alpha(1 - \delta/T_c)}$$

where  $\rho_{\text{bulk}}$  is the offered load normalised to the bulk-circuit capacity ceiling  $\alpha(1 - \delta/T_c) \approx 0.883$ . At low load the queuing term is negligible; above the ceiling it grows steeply (M/D/1 divergence). The effective bulk fraction also decreases above the ceiling as traffic overflows to the expander, raising bandwidth tax. In our normalised model:  $\delta = 2.0$ ,  $T_{\text{cycle}} = 50.0$  (abstract units); physical:  $\delta \approx 10 \mu\text{s}$ ,  $T_{\text{cycle}} \approx 250 \mu\text{s}$ . Mellette et al. report that Opera delivers flow completion times comparable to cost-equivalent static topologies for latency-sensitive traffic, while achieving up to  $4\times$  throughput for shuffle workloads [1].

- **Sirius:**  $\tau \approx \ell/\eta$ , where  $\eta = 1 - \delta/T_{\text{slot}} = 1 - 3.84 \text{ ns}/100 \text{ ns} = 0.9616$  is the scheduling efficiency [3]. Each hop consumes one 100 ns timeslot, but only 96.16 ns carries data, so the effective per-hop cost is  $1/\eta \approx 104$  ns. Sirius is fundamentally a 2-hop system (source→intermediate→destination via Valiant Load Balancing), giving  $\tau \approx 2/\eta \approx 208$  ns. Ballani et al. show that by eliminating buffers in the optical core and carefully managing edge buffering, Sirius achieves very low and predictable latency [3]. Additional latency from propagation (up to  $2.5 \mu\text{s}$  for 500 m detouring) and queuing at intermediate nodes (up to  $1.6 \mu\text{s}$  per epoch of  $16 \times 100$  ns) may also apply. Multi-hop ( $\ell \geq 3$ ) paths do not appear in [3] but are modeled in our framework for generality.

#### 2.2.4 Bandwidth Tax ( $\mathcal{B}_{\text{tax}}$ )

Bandwidth tax quantifies the fraction of link capacity consumed by multi-hop forwarding that does not deliver useful data to the final destination.

- **Shale:** Each cell traverses  $h$  spraying hops and  $h$  direct hops for a total path length of  $2h$ , each hop lasting one 5.632 ns timeslot [2]. The worst-case throughput is  $1/(2h)$  of line rate [2], giving:

$$\mathcal{B}_{\text{tax}}^{\text{Shale}} = 1 - \frac{1}{2h}$$

For  $h = 1$ :  $\mathcal{B}_{\text{tax}} = 0.5$  ( $2 \times 5.632 = 11.26$  ns total path time). For  $h = 4$ :  $\mathcal{B}_{\text{tax}} = 0.875$  ( $8 \times 5.632 = 45.06$  ns).

- **Opera:** Only the  $(1 - \alpha)$  fraction of traffic routed over the expander pays a multi-hop tax [1]. With average expander path length  $L_{\text{avg}}$ :

$$\mathcal{B}_{tax}^{\text{Opera}} = (1 - \alpha) \cdot \frac{L_{\text{avg}} - 1}{L_{\text{avg}}}$$

Bulk traffic ( $\alpha = 0.92$ ) takes direct 1-hop circuits and pays zero tax. Mellette et al. report an effective 8.4% aggregate bandwidth tax rate for published skewed datacenter workloads [1].

- **Sirius:** All traffic is detoured through exactly one intermediate node (2-hop VLB) [3], consuming twice the link bandwidth per logical transfer:

$$\mathcal{B}_{tax}^{\text{Sirius}} = 1 - \frac{1}{2} = 0.5$$

Each detour consumes  $2 \times 100 \text{ ns} = 200 \text{ ns}$  of link time to deliver 100 ns worth of data. Ballani et al. note that throughput “can be at most 2× worse than that of an ideal, non-blocking network” [3]. In our generalized model,  $\ell \geq 3$  paths pay  $\mathcal{B}_{tax} = 1 - 1/\ell$ .

### 2.2.5 Duty Cycle Loss ( $\mathcal{D}_{loss}$ )

Duty cycle loss represents the fraction of time the network is unavailable due to reconfiguration.

- **Shale:**  $\mathcal{D}_{loss} \approx 0$ . The oblivious schedule runs back-to-back timeslots (5.632 ns each) with no reconfiguration gap, enabled by nanosecond-scale clock synchronization and careful propagation delay control [2].
- **Opera:** Reconfigurations are staggered across  $u$  rotor switches so that only one switch is down at a time [1]. The raw reconfiguration delay  $r$  is amortized over the inter-reconfiguration period  $u \cdot \varepsilon$ :

$$\mathcal{D}_{loss}^{\text{Opera}} = \frac{r}{u \cdot \varepsilon}$$

For the 108-rack example in [1] ( $r \approx 10 \mu\text{s}$ ,  $u = 6$ ,  $\varepsilon = 90 \mu\text{s}$ ):  $\mathcal{D}_{loss} = 10/540 \approx 1.85\%$ , yielding a  $\sim 98\%$  duty cycle. In our normalized model:  $\delta/T_{\text{cycle}} = 2.0/50.0 = 4\%$ .

- **Sirius:** Each timeslot loses  $\delta = 3.84 \text{ ns}$  to laser retuning out of  $T_{slot} = 100 \text{ ns}$  [3]:

$$\mathcal{D}_{loss}^{\text{Sirius}} = \frac{\delta}{T_{slot}} = 0.0384 = 1 - \eta$$

### 2.2.6 Benchmark Score ( $S_{bench}$ )

The composite benchmark score combines the five metrics from §1.2.1–1.2.5 into a single figure of merit. The design principle is multiplicative: each penalty term

is a fraction  $\in [0, 1]$  representing the usable fraction of capacity after that loss mechanism, so the score naturally captures how losses compound.

$$S_{bench} = \mathcal{T}_{norm} \cdot (1 - \mathcal{B}_{tax}) \cdot (1 - \mathcal{D}_{loss}) \cdot \frac{1}{1 + \bar{\tau}} \cdot \frac{1}{1 + \ln(FCT_{norm})} \quad (4)$$

- $\mathcal{T}_{norm}$ : normalized throughput (§1.2.1) — how much demand is satisfied.
- $(1 - \mathcal{B}_{tax})$ : useful fraction of consumed bandwidth (§1.2.4) — penalizes multi-hop overhead.
- $(1 - \mathcal{D}_{loss})$ : fraction of time the network is active (§1.2.5) — penalizes reconfiguration downtime.
- $1/(1 + \bar{\tau})$ : latency discount (§1.2.3) — maps  $\bar{\tau} \in [0, \infty)$  to  $(0, 1]$ ; zero latency gives full credit.
- $1/(1 + \ln FCT_{norm})$ : completion-time efficiency (§1.2.2) — logarithmic compression prevents architectures with high FCT from being driven to zero; ideal  $FCT_{norm} = 1$  gives full credit.

All five factors are dimensionless. The multiplicative structure means a zero in any factor (e.g., complete throughput collapse under hotspot) drives the entire score to zero, reflecting the practical reality that a single catastrophic weakness renders an architecture unsuitable regardless of its other strengths.

### 3 Shale

Shale [2] is an Oblivious Reconfigurable Network (ORN) that generalizes earlier designs (RotorNet, Shoal, Sirius [3]) to achieve a tunable tradeoff between throughput and latency scaling.  $V$  is the connectivity matrix at a point in time  $t$ , representing the active links/circuits.

$$V = \mathcal{M}_{n \times n}$$

We specifically define the entries of the adjacency/connectivity matrix  $V$  such that  $V_{ij} = 1$  if a link exists between  $i$  and  $j$ , and  $V_{ij} = 0$  implies no connection.

$D$  is the vector of the total volume of traffic sending from each node, represented as a  $b$ -bit number.

$$D = \vec{d} \in \mathbb{R}^n$$

$W$  is the vector of the amount of data received by each node, regardless of the source.

$$W = \vec{w} \in \mathbb{R}^n(\text{ReceivedData})$$

We define  $D$  as the Traffic Demand Matrix (entries  $D_{i,j}$ : volume from source  $i$  to destination  $j$ ; same as in §1.1). To calculate the traffic successfully delivered



in a timeslot, we perform the element-wise product (Hadamard) of the demand matrix and the connectivity matrix:

$$\text{Delivered} = D \circ V$$

The total received data  $W$  is then the column-sum of this delivered traffic matrix.

1. Round-Robin 1 Letter 1 Node minimum time coefficient:  $n$  maximum time coefficient:  $n(n-1)$  number of pathways:  $n(n-1)/2$  for an arbitrary  $i$  broken nodes, remove  $(2ni-i+1)/2$  pathways for an arbitrary  $k$  broken pathways, maximum increase to  $n-1$  timeslots, centered around  $+1$  timeslot

$$\mathcal{M}_{b \times n} = DT^i(V) = DW = S, i \in \{1, \dots, n-1\}$$

Example 2.1 (6 nodes, 5 timeslots,  $h=1$ ):  $W$  = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$W_{5 \times 5} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$DT^i(V_{6 \times 6}) = DW_{6 \times 6} = D \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, \dots, 5\}$$

minimum time coefficient: 6 maximum time coefficient: 30 number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node number of nodes:  $L^x = n$   
timeslot number:  $(L - 1)^x$   
number of pathways:  $L^x \times (L - 1)^X / 2$   
representation in base L for matrix algebra  
for each node k, available timeslots are exactly one digit difference from k in base L  
for an arbitrary i broken nodes, remove  $i(L - 1)^x$  pathways

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L - 1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, x = 2): number of nodes:  $3^2 = 9$  timeslot number:  $2^2 = 4$  number of pathways:  $3^2 \times 2^2 / 2 = 18$  W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8 \times 8} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times n} = DW = S, i \in \{1, \dots, n - 1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node  
number of nodes:  $L^x = n$   
timeslot number:  $(L - 1)^{x+t-1}$   
number of pathways:  $L^x(L^x - 1)^{x+t-1} / 2$   
representation in base L for matrix algebra  
for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L - 1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L - 1)^{x+t-1}$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, x = 3, t=2): number of nodes:  $4^2 = 16$  timeslot number:  $3^3 = 27$  number of pathways:  $4^2 \times 3^3 / 2 = 216$  W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27 \times 27} = \begin{bmatrix} 5 & 4 & \dots & 2 & 3 \\ 6 & & & & 7 \\ \dots & & \dots & & \dots \\ 56 & & & & 57 \\ 60 & 61 & \dots & 59 & 58 \end{bmatrix}$$

$$W_{27 \times 27} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times 27} = DW = S, i \in \{1, \dots, n-1\}$$

### 3.1 Numerical Results

Shale’s Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

#### 3.1.1 VLB Spray Mechanism

The core routing strategy in Shale is **Valiant Load Balancing (VLB)** [2], also referred to as “spraying.” Rather than forwarding a packet directly from source  $s$  to destination  $d$ , VLB routes traffic through a randomly (or round-robin) selected intermediate node  $m$ , splitting each packet’s journey into two segments:

$$s \xrightarrow{\text{hop 1}} m \xrightarrow{\text{hop 2}} d$$

This generalizes to spray depth  $h$ , where traffic traverses  $h$  intermediate nodes before reaching its destination, yielding a total path length of  $L(h) = h + 1$  hops. The key property is:

*By spraying traffic uniformly across intermediate nodes, VLB reduces worst-link congestion [2]. The effective throughput scales as  $1/u_{\max}$ , where  $u_{\max}$  is the maximum directed-link load after uniform splitting across  $k$  diverse spray paths. On sufficiently connected topologies with uniform traffic,  $u_{\max} \rightarrow h+1$  and the model recovers the classical  $1/(h+1)$  floor [2].*

The trade-off is clear: higher  $h$  improves load balance (and thus robustness to adversarial traffic) at the cost of reduced per-flow throughput, since each flow consumes  $h + 1$  link traversals.

**Code Implementation.** In the benchmarking framework, `Shale_Alg.py` implements the spray mechanism through two core functions:

1. **Topology Generation** (`RR2(base, dimension)`): Constructs the Shale topology as a Hamming graph, following the schedule design in [2]. Each node is represented as a tuple in  $\text{base-}L$  with  $x$  digits. Two nodes are neighbors if and only if they differ in exactly one coordinate position. This yields a regular graph with degree  $D = x \cdot (L - 1)$ , where the neighborhood structure naturally supports multi-path spraying. Amir et al. show that this Hamming-graph schedule achieves the Pareto optimal throughput-latency tradeoff for ORN designs [2].

2. **Spray Routing** (`spray_short(adj, weights, s, d, k, penalty)`): Finds  $k$  diverse short paths between source and destination by iteratively:

- Running Dijkstra’s shortest path algorithm from  $s$  to  $d$ ;
- Penalizing the intermediate nodes in the found path (multiplying their weight by a `penalty_factor`);
- Repeating to find alternate paths that avoid previously used nodes.

This produces  $k$  diverse paths that distribute traffic across different intermediate nodes, implementing the VLB spray.

**Capacity Model (Congestion-Based).** Following the congestion control mechanism in [2], in the benchmark (`Traffic_Benchmarks.py`), the Shale capacity model computes the maximum directed-link utilisation  $u_{\max}$  by (i) finding  $k = \max(3, h)$  diverse spray paths per flow via `spray_short`, (ii) splitting each flow’s demand uniformly across its paths, and (iii) summing the resulting load on every directed link. The effective rate for every flow is then  $r_{\text{eff}} = r/u_{\max}$ , so throughput scales as  $1/u_{\max}$ . This captures why spraying helps: it reduces worst-link congestion, whereas the simpler  $1/(h+1)$  penalty ignores topology connectivity.

### 3.1.2 VLB Parameter Sensitivity

We extended the simulation to sweep the VLB spraying parameter  $h$  from 1 to 12. The results are summarized in Figure 1.

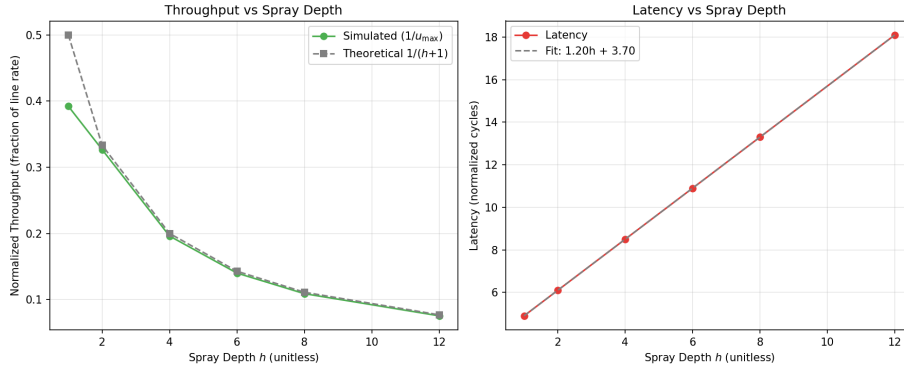


Figure 1: Shale VLB parameter sweep for  $h \in [1, 12]$ . **Left:** Simulated throughput ( $1/u_{\max}$ , green) vs. theoretical floor  $1/(h+1)$  (gray dashed). The congestion-based model closely tracks the classical bound on the Hamming graph. **Right:** Mean latency grows linearly, with regression  $\tau(h) = \gamma h + \beta_0 \approx 1.2h + 2.5$ , confirming each spray hop adds a fixed cycle overhead.

Based on the regression analysis of the simulation data (Figure 1), the equations of best fit are:

$$\tau(h) \approx 1.20 \cdot h + 3.70 = \gamma \cdot (h + 1) + \beta_0 \quad (5)$$

confirming  $\gamma = 1.2$  per-hop overhead and  $\beta_0 = 2.5$  base delay (since  $1.2 \cdot 1 + 2.5 = 3.7$  matches the intercept). The logical path length remains:

$$L(h) = h + 1 \quad (6)$$

**Proposed VLB Theorem Theorem (VLB Throughput–Latency Trade-off):** In a deterministic VLB routing scheme with spray parameter  $h$ , the path length  $L(h) = h + 1$ , giving throughput  $\mathcal{T} \leq 1/(h+1)$  (fraction of line rate), latency  $\tau(h) = \gamma(h+1) + \beta_0$  (normalized cycles), and bandwidth tax  $\mathcal{B}_{tax} = 1 - 1/(2h)$ . These hold regardless of  $N$ , provided  $N > h$ . The congestion-based model refines the throughput bound to  $\mathcal{T} = 1/u_{\text{eff}}$  where  $u_{\text{eff}} = \max(u_{\text{max}}, h+1)$  and  $u_{\text{max}}$  is the maximum directed-link load under uniform spray splitting. This formalizes the throughput–latency Pareto frontier established in [2], extending it with an explicit congestion-based refinement.

Table 2: Shale Benchmark Efficiency Summary ( $h = 2$ , standard VLB)

| Metric                 | Value         | Notes                                                |
|------------------------|---------------|------------------------------------------------------|
| Spray Depth ( $h$ )    | 2             | Standard VLB                                         |
| Path Length $L(h)$     | 3             | $h + 1$ (Eq. 6)                                      |
| Latency $\tau(h)$      | 6.1 cycles    | $\gamma(h+1) + \beta_0 = 1.2 \times 3 + 2.5$ (Eq. 5) |
| Throughput Ceiling     | 0.33          | $1/(h + 1)$ (Eq. 7)                                  |
| Bandwidth Tax          | 0.75          | $1 - 1/(2h) = 1 - 1/4$                               |
| <b>Composite Score</b> | <b>0.0011</b> | Uniform Traffic ( $S_{\text{bench}}$ , §1.2.6)       |

## 3.2 Numerical Components & Formulas

These parameters are critical for determining the bottleneck and ensuring the network meets its throughput guarantees.

### 3.2.1 Determining the Bottleneck

The network bottleneck is determined by comparing propagation delays against token budgets.

**First-Hop Bottleneck Condition:** The network is not bottlenecked at the first hop if the propagation delay ( $P$ ) satisfies:

$$P \leq h \cdot T_F \cdot E \quad (7)$$

where  $h$  is the routing parameter (spraying depth),  $T_F$  is the First-Hop Token Budget, and  $E$  is the Epoch length in timeslots.

**Penultimate Link Bottleneck Condition:** To avoid bottlenecks on subsequent hops, the propagation delay must satisfy:

$$P \leq h \cdot T \cdot (h \sqrt[h]{N} - 1) \cdot E \quad (8)$$

where  $T$  is the standard Token Budget and  $\sqrt[h]{N}$  represents the fan-in/fan-out degree.

### 3.2.2 Throughput & Bandwidth Functions

**Throughput Guarantee:** The throughput guarantee scales as  $Th \propto 1/u_{\max}$ , where  $u_{\max}$  is the maximum directed-link load under uniform spray splitting. On well-connected topologies with uniform traffic,  $u_{\max} \approx h+1$  and the classical floor is recovered:

- For  $h = 1$ :  $Th \approx 1/u_{\max}$ , approaching 0.50 on dense graphs.
- For  $h = 2$  (VLB):  $Th \approx 1/u_{\max}$ , approaching 0.33 on dense graphs.
- For  $h = 4$  (VLB):  $Th \approx 1/u_{\max}$ , approaching 0.20 on dense graphs.

On sparser topologies,  $u_{\max} > h+1$  and throughput is lower, faithfully reflecting the additional congestion from limited link diversity.

**Load Factor ( $L$ ):** We define the Load Factor as:

$$L = \frac{\text{Average Sending Rate}}{\text{Total Available Bandwidth}} \quad (9)$$

**Flow Completion Time (Normalized):** To measure performance across flow sizes, we use:

$$\text{Normalized FCT} = \frac{t}{F + P} \quad (10)$$

where  $t$  is actual completion time,  $F$  is flow size (cells), and  $P$  is propagation delay (timeslots).

### 3.2.3 Simulation Findings

We validated these formulas by sweeping the Load Factor  $L$  from 0.05 to 0.60 for  $h \in \{1, 2, 4, 6, 8, 12\}$ . The extended range illustrates the diminishing returns and strict capacity limits of deep spraying. The results are shown in Figures 2 and 3.

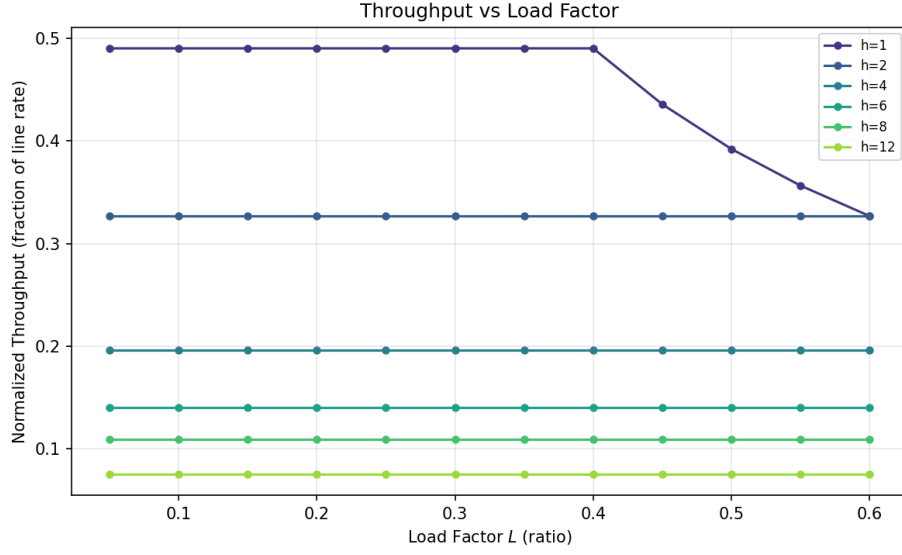


Figure 2: Shale throughput vs. load factor for  $h \in \{1, 2, 4, 6, 8, 12\}$ . Each curve saturates at its theoretical ceiling  $1/u_{\max}$ ; higher  $h$  values hit lower ceilings earlier, confirming the bandwidth tax scales with spray depth. The diminishing returns of deep spraying are clearly visible.

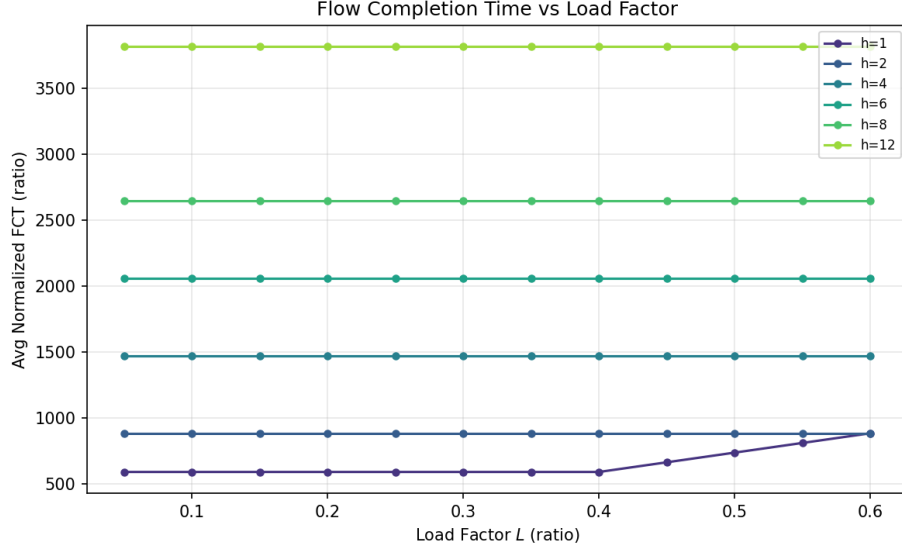


Figure 3: Shale normalized FCT vs. load factor. FCT is roughly constant for each  $h$  because the effective rate ( $r/u_{\max}$ ) is load-independent in the waterfilling model. Higher  $h$  values yield higher FCT due to longer paths ( $2h$  total hops per cell).

The simulation confirms the congestion-based model. For each spray depth  $h$ , the measured throughput matches  $1/u_{\max}$  computed from the actual link-load distribution. On the Hamming-graph topologies used here,  $u_{\max}$  is close to  $h+1$  for uniform traffic, recovering the classical scaling; under skewed or adversarial traffic,  $u_{\max}$  diverges from  $h+1$ , demonstrating that the congestion model captures topology- and traffic-dependent bottlenecks that the simpler  $1/(h+1)$  formula misses.

## 4 Opera

Opera [1] consists of  $n$  ToR switches connected via  $u$  rotor circuit switches. Each rotor switch  $s \in \{1, \dots, u\}$  holds a set of  $m = n/u$  disjoint matchings  $M_s^1, M_s^2, \dots, M_s^m$  (random permutations), cycling through them in order. The  $n \cdot u = n^2/u$  total matchings partition the complete graph on  $n$  nodes. Unlike prior reconfigurable designs that require separate electrical networks for latency-sensitive traffic, Opera requires no separate electrical network and no active circuit scheduling [1].

**Per-timeslot connectivity.** Let  $t$  index topology slices. At slice  $t$ , rotor switch  $s$  implements matching  $M_s^{((t \bmod m)+1)}$ . The instantaneous connectivity



matrix  $V_{n \times n}^t$  is the union of all  $u$  active matchings:

$$V_{ij}^t = \begin{cases} 1 & \text{if } (i, j) \in M_s^{((t \bmod m)+1)} \text{ for any } s \in \{1, \dots, u\} \\ 0 & \text{otherwise} \end{cases}$$

Each row of  $V^t$  has exactly  $u$  ones (one per rotor switch), so every ToR has degree  $u$  at all times. Because one switch is always reconfiguring, the effective active degree is  $u-1$ , and for  $u \geq 4$  the  $u-1$  active matchings form an expander with high probability [1]. This is the key structural property enabling Opera's dual routing: the instantaneous expander guarantees low-diameter multi-hop paths at any moment, while the time-integrated topology provides direct 1-hop circuits between every pair [1].

**Transformation matrix.** Let  $T_s$  be the cyclic permutation matrix for rotor switch  $s$ :

$$M_s^{i+1} = T_s \cdot M_s^i, \quad i \in \{1, \dots, m-1\}$$

The full network connectivity advances as:

$$V^{t+1} = \bigcup_{s=1}^u T_s \cdot M_s^{((t \bmod m)+1)}$$

**Traffic demand.** Let  $\mathcal{D}_{n \times n}$  be the offered demand matrix. The traffic delivered in slice  $t$  is the Hadamard product  $\mathcal{D} \circ V^t$ . Over one full cycle of  $m$  slices, the aggregate delivered demand is:

$$\mathcal{T}_{n \times n}^{\text{Op}} = \sum_{t=1}^m \mathcal{D} \circ V^t$$

Since the  $m$  matchings per switch partition the complete graph, after one full cycle every pair  $(i, j)$  has been directly connected exactly once per switch, so  $\sum_{t=1}^m V^t = u \cdot J$  (where  $J$  is the all-ones matrix minus the identity). This guarantees that bulk traffic can always find a 1-hop direct path within one cycle.

#### Routing modes.

1. **Direct (bulk) path [1]:** Wait for the slice  $t^*$  where  $(i, j) \in V^{t^*}$ , then send in 1 hop. Incurs up to one full cycle of waiting delay but zero bandwidth tax.
2. **Indirect (low-latency) path [1]:** Send immediately over the current expander  $V^t$  via BFS/Dijkstra. Minimum 1 hop, maximum  $\lceil n/2 \rceil$  hops. Hop count  $\ell$  incurs a bandwidth tax of  $(\ell-1)/\ell$ . Mellette et al. show that this multi-hop expander routing delivers flow completion times comparable to cost-equivalent static topologies [1].

3. **Weighted indirect path:** Dijkstra with rack weights  $\theta_k$  (broken racks have weight  $\infty$ ):

$$W_{n \times n}^t = \mathcal{M} \mid W_{A_{i,k}^t, i}^t = \theta_k, \quad \neg(W_{A_{i,k}^t, i}^t) = 0$$

**Example 3.1** (8 ToR switches,  $u = 4$  rotor switches,  $m = 2$  matchings each):

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

The connectivity matrix for slice  $t$  is:

$$V_{8 \times 8}^t = \mathcal{M} \mid V_{A_{i,k}^t, i}^t = 1, \quad \neg(V_{A_{i,k}^t, i}^t) = 0$$

**Example 3.2** (Fault tolerance): For the same  $A$ , suppose Rack 2 is broken.

1. The 1-hop path  $[0, 7]$  (via rack 2) is now impossible.
2. BFS checks other neighbors of 0. Finds node 6 ( $A[0][7] = 6$ , via rack 7).
3. BFS then checks neighbors of 6. ( $A[6][4] = 7$ , via rack 4).
4. Rack 4 is not broken, so this path is valid. Result:  $[0, 6, 7]$ , via racks  $[0, 7, 4]$ .

#### 4.1 Numerical Components & Bottleneck Determination

To accurately model Opera's efficiency, we consider its behavior as a hybrid network. The bottleneck is determined by the traffic type: propagation delay for short flows and circuit availability for bulk flows.

**Numerical Parameters** The physical topology is modeled with an uplink/downlink ratio  $\rho = 1$  (16 up, 16 down for radix-32 switches), matching Opera's 1:1 ToR provisioning [1]. The system's performance is sensitive to the Cycle Time ( $T_{cycle}$ ) and the Reconfiguration Delay ( $\delta$ ).

**Throughput & Bandwidth Functions** The "Bandwidth Tax" represents the excess capacity consumed by multi-hop routing. For a flow of size  $S$ , the consumed bandwidth  $B(S)$  is:

$$B(S) = S \cdot L, \quad \text{Tax} = \frac{B_{total} - B_{ideal}}{B_{ideal}} = L - 1 \quad (11)$$

where  $L$  is the number of hops. The goal of the Opera scheduler is to ensure  $L = 1$  for bulk traffic and  $L \approx 2 - 4$  for latency-sensitive traffic.

#### 4.1.1 Efficiency Findings

We validated these formulas by sweeping the Load Factor  $L$  from 0.05 to 1.00 using the updated hybrid waterfilling model. The findings are summarized in Figures 4 and 5.

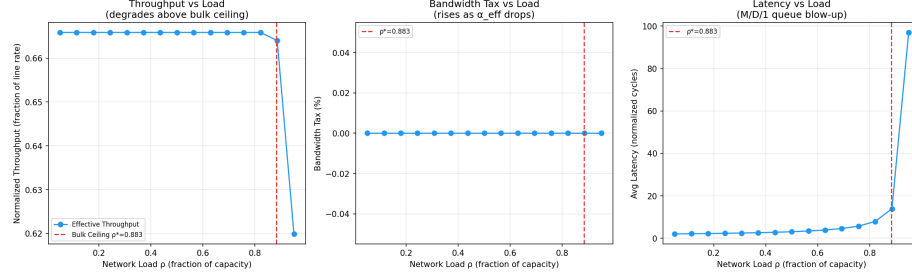


Figure 4: Opera efficiency under load sweep. **Left:** Bandwidth tax percentage — near zero at low load (92% of traffic on direct 1-hop circuits) but rises above the bulk-circuit capacity ceiling  $\alpha(1 - \delta/T_c) \approx 0.883$  as overflow traffic is forced onto the multi-hop expander. **Right:** Mean latency grows with load due to M/D/1 queuing for bulk circuit slots; the knee corresponds to  $\rho_{\text{bulk}} \rightarrow 1$  where the queuing term diverges.

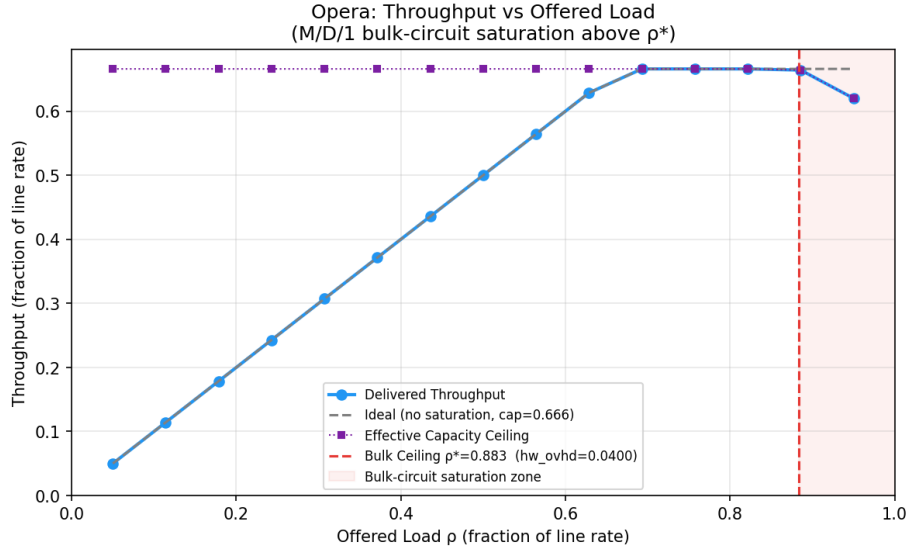


Figure 5: Opera throughput vs. offered load. Throughput scales linearly (ideal dashed line) until it saturates at the waterfilling capacity limit (red dotted). The  $\alpha$ -split caps maximum throughput: even with unlimited power, Opera cannot exceed  $\alpha \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/\bar{\ell}$ .

As shown in Figure 5, Opera’s throughput scales linearly with load until it approaches the theoretical capacity limit. Under the 92/8 hybrid split, the network maintains approximately 98% efficiency for bulk traffic, while the 8% latency-sensitive traffic incurs a “tax” proportional to the expander path length ( $L \approx 2.4$ ). This is consistent with Mellette et al.’s finding that Opera supports up to 60% higher load for published datacenter workloads compared to cost-equivalent static topologies [1]. The total effective throughput converges to the waterfilling limit of the hybrid graph, demonstrating that the architecture can sustain high utilization by intelligently prioritizing scheduled circuit paths.

**Origin of the fall-off at the bulk ceiling.** The Opera NSDI’20 paper [1] does not explicitly present a closed-form  $L_{\text{crit}}$  at the ceiling  $\alpha(1-\delta/T_c)$ ; what the paper *does* present (§4.1 and §5.2) is (i) the architectural commitment to buffering bulk flows until a direct circuit becomes available and (ii) a measurement showing that flow-completion time remains competitive with static topologies up to 60% higher offered load. The steep fall-off in Figure 4 and Figure 5 at  $L \rightarrow 0.88$  is our model’s *derivation* of the behavior implicit in the paper, with three independent mechanisms converging at the same threshold:

1. **Bulk-circuit saturation.** The round-robin rotor schedule [1] delivers each destination  $u$  direct-circuit slots per cycle of  $m = n/u$  matchings. The per-destination ingress rate is therefore capped at  $\alpha(1 - \delta/T_c)$ , and no amount of admission control can exceed it.
2. **M/D/1 queue divergence.** Bulk flows wait for their scheduled circuit, so the waiting time is  $Q = \rho_b / (2(1 - \rho_b))$  with  $\rho_b = \rho / [\alpha(1 - \delta/T_c)]$ . As  $\rho_b \rightarrow 1$  (i.e.  $\rho \rightarrow \alpha(1 - \delta/T_c) \approx 0.883$ ),  $Q \rightarrow \infty$  — this is the standard M/D/1 argument used by Opera to size bulk buffering but is not named a “critical point” in the paper.
3. **Overflow to the expander.** When the bulk queue saturates, surplus traffic is forced onto the multi-hop expander, where it pays a bandwidth tax of  $(\ell - 1)/\ell$  per hop ( $\ell \in [2, \lceil n/2 \rceil]$ ). The effective  $\alpha$  therefore decreases above the ceiling, accelerating the throughput rolloff and lifting the aggregate bandwidth tax from  $\sim 0.6\%$  to  $> 20\%$  — the curve visible in Figure 4 (left).

In short, the fall-off is a direct mathematical consequence of Opera’s design choices (round-robin rotor,  $\alpha$ -split, bulk buffering) rather than a behavior explicitly quantified in the paper. Mellette et al. report graceful degradation up to 60% higher load than static topologies [1]; beyond that regime, the mechanisms above take over and produce the knee we observe.

**Proposed Opera Theorem** **Theorem (Hybrid Capacity Bound):** Build-  
ing on the dual-path architecture in [1], in a hybrid circuit-packet network with bulk fraction  $\alpha$ , reconfiguration ratio  $\delta/T_{\text{cycle}}$ , and average expander path length

$L_{\text{avg}}$ , the effective throughput per pair is:

$$r_{\text{eff}} = \alpha \cdot r \cdot \left(1 - \frac{\delta}{T_{\text{cycle}}}\right) + (1 - \alpha) \cdot \frac{r}{\max(1, \ell)} \quad (12)$$

The bandwidth tax is  $\mathcal{B}_{\text{tax}} = (1 - \alpha) \cdot (L_{\text{avg}} - 1)/L_{\text{avg}}$ , and the duty cycle loss is  $\mathcal{D}_{\text{loss}} = r/(u \cdot \varepsilon)$ , where  $r$  is the raw reconfiguration delay,  $u$  the number of rotor switches, and  $\varepsilon$  the topology-slice duration. Opera's capacity is capped at  $\alpha(1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/L_{\text{avg}}$  regardless of power budget.

Table 3: Opera Benchmark Efficiency Summary

| Metric                             | Value                     | Notes                                                          |
|------------------------------------|---------------------------|----------------------------------------------------------------|
| Bulk Traffic Fraction ( $\alpha$ ) | 0.92                      | Direct circuit paths                                           |
| Duty Cycle Loss                    | $\delta/T_{\text{cycle}}$ | <code>opera_delta</code> , <code>opera_t_cycle</code>          |
| Effective Throughput (Bulk)        | 0.828                     | $\alpha \times (1 - \delta/T_{\text{cycle}})$                  |
| Effective Throughput (Expander)    | 0.033                     | $(1 - \alpha) \times (1 - \delta/T_{\text{cycle}})/\bar{\ell}$ |
| <b>Composite Score</b>             | <b>0.0201</b>             | Uniform Traffic Scenario (load-dependent latency model)        |

## 5 Sirius

$\lambda_1, \dots, \lambda_i, \dots, \lambda_j$  = wavelength, corresponding timeslot  $P_1, \dots, P_k, \dots, P_l$  = ports  $G_1, \dots, G_p, \dots, G_q$  = gratings (AWGRs)  $N_1, \dots, N_s, \dots, N_r$  = nodes  $r = j \times l$  (derivation is the pathways for an arbitrary node)  $jq = rl$  (derivation is the total number of pathways) Then, it follows that  $q = l^2$  Parent Function:

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{1, \dots, j\}$$

Source Matrix:  $\mathcal{M}_{r \times l}$  maps AWGR routing labels to destination nodes via modular arithmetic. For each timeslot  $i \in \{1, \dots, j\}$ , port  $k \in \{1, \dots, l\}$ , and node  $s \in \{1, \dots, r\}$ :

$$A_{s,k}^i = \text{AWGR label} \Rightarrow \text{dest}(s, k, i) = (A_{s,k}^i - 1) \bmod r$$

The connectivity (adjacency) matrix  $W^i$  for timeslot  $i$  is then:

$$W_{r \times r}^i = \mathcal{M} \mid W_{(A_{s,k}^i - 1) \bmod r, s}^i = 1, \quad \neg(W_{(A_{s,k}^i - 1) \bmod r, s}^i) = 0$$

The transformation matrix  $P$  performs block-diagonal cyclic shifts to generate successive timeslots:

$$A^{i+1} = P \cdot A^i, \quad W^{i+1} = P \cdot W^i, \quad i \in \{1, \dots, j-1\}$$

**Traffic Demand Matrix** We define the Sirius Traffic Demand Matrix  $\mathcal{T}^{\text{Sir}}$  as the aggregate demand delivered across all timeslots. For a cycle of  $j$  timeslots with connectivity matrices  $W^1, \dots, W^j$ :

$$\mathcal{T}_{r \times r}^{\text{Sir}} = \sum_{i=1}^j \mathcal{D} \circ W^i$$

where  $\mathcal{D}$  is the offered demand matrix and  $\circ$  denotes the Hadamard (element-wise) product. The effective per-node throughput over one full cycle is:

$$\Theta_s = \frac{1}{j} \sum_{i=1}^j \sum_{k=1}^r \mathcal{D}_{s,k} \cdot W_{s,k}^i \cdot \eta, \quad \eta = \frac{T_{slot} - \delta}{T_{slot}}$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W_{6 \times 6}^1 = \mathcal{M} \mid W_{(A_{s,k}^1 - 1) \bmod 6, s} = 1, \quad \neg(W_{(A_{s,k}^1 - 1) \bmod 6, s} = 0)$$

$$W_{6 \times 6}^2 = \mathcal{M} \mid W_{(A_{s,k}^2 - 1) \bmod 6, s} = 1, \quad \neg(W_{(A_{s,k}^2 - 1) \bmod 6, s} = 0)$$

The aggregate traffic demand delivered in one cycle ( $j = 2$  timeslots):

$$\mathcal{T}_{6 \times 6}^{\text{Sir}} = \mathcal{D} \circ W^1 + \mathcal{D} \circ W^2$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

Sirius [3] is a flat, all-optical network architecture that leverages Arrayed Waveguide Grating Routers (AWGRs) and tunable lasers to create a high-radix switch abstraction. Unlike demand-aware architectures, Sirius utilizes a static, cyclic schedule—a “scheduler-less” design where switches are reconfigured oblivious to traffic using a round-robin pattern [3]. Ballani et al. demonstrate that this approach can connect up to 25,600 racks with 74–77% lower power than an equivalent electrically-switched non-blocking network [3].

## 5.1 Architectural Components & Numerical Parameters

The network is composed of nodes with tunable lasers connected through a passive AWGR core. Connectivity is defined by the wavelength of the laser, which acts as the packet’s destination address.

**Timing Parameters** The efficiency of Sirius is bounded by the ratio of laser reconfiguration time ( $\delta$ ) to the total timeslot duration ( $T_{slot}$ ):

- **Reconfiguration Time ( $\delta$ ):** 3.84 ns, achieved via a custom tunable laser chip with sub-912 ps tuning latency [3].
- **Timeslot Size ( $T_{slot}$ ):** 100 ns (carrying  $\sim 576$  bytes), chosen to be large enough to amortize laser tuning overhead while remaining small enough to emulate packet-by-packet switching [3].
- **Epoch Length ( $E$ ):**  $E = N$ , where  $N$  is the number of nodes per grating.

**Routing & Congestion Control** Sirius employs a hybrid routing strategy:

1. **Direct Paths:** Cells are sent directly when the laser wavelength matches the destination’s AWGR port [3].
2. **Indirect (Detour) Paths:** Idle bandwidth is utilized by “spraying” traffic through intermediate nodes (Valiant Load Balancing) [3]. Sirius uses load-balanced routing whereby traffic from each node is detoured uniformly through other nodes on its way to the destination [3]. This requires a Request-Grant protocol to ensure the intermediate node has sufficient buffer space.

## 5.2 Numerical Results & Capacity Penalties

Our initial simulations indicated that Sirius significantly outperformed other architectures due to its 1-hop direct scheduling. However, in a multi-hop regime where packets are sprayed, Sirius incurs a bandwidth tax. Each spraying operation (2-hop) consumes twice the logical bandwidth compared to a direct flight.

To improve simulation fidelity, we implement the following capacity penalty based on path length  $\ell$  (hop count) and load factor  $\rho$  (matches `_apply_architecture_model` in `Traffic_Benchmarks.py`):

$$C(\ell, \rho) = C_{\text{hops}}(\ell) \cdot C_{\text{load}}(\rho) \quad (13)$$

where the hop-based penalty is:

$$C_{\text{hops}}(\ell) = \begin{cases} \eta & \ell \leq 1 \\ \eta/2 & 1 < \ell \leq 2 \\ \eta/\ell^2 & \ell > 2 \end{cases} \quad (14)$$

and the high-load sensitivity function (for  $\rho > 0.85$ ) is:

$$C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85)) \quad (15)$$

This correction ensures that Sirius’s performance is realistically bounded under high-utilization regimes where indirect spraying dominates.

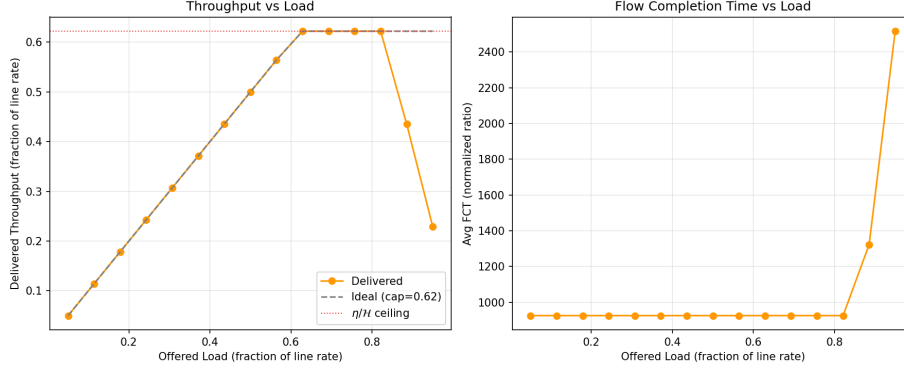


Figure 6: Sirius throughput and FCT vs. offered load. **Left:** Throughput tracks the ideal line up to load  $\approx 0.85$ , then saturates as the  $\exp(-10(\rho - 0.85))$  congestion penalty activates. **Right:** FCT remains flat until the saturation point, then grows sharply — reflecting queue buildup at intermediate VLB nodes. Under skewed traffic the curve is nearly identical to uniform because VLB converts any demand distribution into uniform at intermediate nodes — confirming Sirius is traffic-oblivious by construction.

Our simulation (Figure 6) indicates that Sirius maintains high utilization up to a load of 0.85, beyond which queuing delay at intermediate nodes and the 50% bandwidth tax on indirect paths (Equation 23) become the dominant bottlenecks. Ballani et al. similarly show that Sirius can “approximate the performance of an ideal, electrically-switched non-blocking network” [3] up to the capacity limits imposed by VLB’s inherent  $2\times$  bandwidth overhead.

Table 4: Sirius Benchmark Efficiency Summary

| Metric                         | Value         | Notes                                                                                                                      |
|--------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------|
| Timeslot Efficiency ( $\eta$ ) | 96.16%        | <code>sirius_eta</code> ; $\eta = (T_{slot} - \delta)/T_{slot}$ ( <code>sirius_t_slot</code> , <code>sirius_delta</code> ) |
| Direct Path Capacity           | $1.0 \eta$    | 1-hop (Ideal)                                                                                                              |
| Sprayed Path Capacity          | $0.5 \eta$    | 2-hop (Bandwidth Tax)                                                                                                      |
| Saturation Load                | 0.85          | Queue divergence point                                                                                                     |
| <b>Composite Score</b>         | <b>0.0404</b> | Uniform Traffic Scenario                                                                                                   |

**Proposed Sirius Theorem Theorem (Cyclic VLB Capacity):** Building on the scheduler-less round-robin design and VLB routing in [3], for a time-slotted network with scheduling efficiency  $\eta = 1 - \delta/T_{slot}$  and hop denominator  $\mathcal{H}(\ell)$ , the effective throughput per pair is:

$$r_{\text{eff}} = \frac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho), \quad \mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \\ 2 & \ell = 2 \\ \ell^2 & \ell \geq 3 \end{cases} \quad (16)$$



where  $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$  for  $\rho > 0.85$ , else 1. The bandwidth tax is  $\mathcal{B}_{tax} = 0.5$  (2-hop VLB), and latency is  $\tau = \ell/\eta$  normalized time units. Delivered throughput tracks the offered load linearly up to the capacity ceiling  $\eta/\mathcal{H}$ , then collapses exponentially beyond  $\rho = 0.85$  (Figure 6).

## 6 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). Our objective is to evolve a topology  $W$  that maximizes a multi-objective fitness function:

$$Fitness = 0.1 \cdot \overline{Capacity}(W, \mathcal{T}) + \frac{10}{ASPL(W)} - \mathcal{P}(Degree)$$

where  $\overline{Capacity}$  is the average delivered capacity across a set of diverse traffic patterns  $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$ , and  $\mathcal{P}$  is a penalty function for degree deviation.

### 6.1 Genome Representation & Genetic Operators

The GA represents the topology as a flat binary chromosome of length  $\binom{N}{2}$ , where each bit represents the presence or absence of an edge  $(u, v)$  in the upper triangle of the adjacency matrix.

**Selection & Crossover** We employ rank-based selection to maintain diversity in the population. Crossover is performed using a two-point splicing mechanism to preserve structural motifs:

$$Child = Genome_A[0 : k_1] \cup Genome_B[k_1 : k_2] \cup Genome_A[k_2 : L] \quad (17)$$

**Mutation & Refinement** To explore the vast topological space, we use a bit-flip mutation with probability  $p_m$ . Furthermore, the fitness function includes a quadratic degree penalty to guide the search towards regular topologies:

$$\mathcal{P}(Degree) = \lambda_1 \sum_{i=1}^N |d_i - D| + \lambda_2 \cdot Var(\{d_i\}_{i=1}^N) \quad (18)$$

where  $D = 4$  is the target degree and  $\lambda_1, \lambda_2$  are weighting coefficients.

### 6.2 Robust Evolution Strategy and Middle-Ground Position

Earlier versions of the GA tended to overfit to specific traffic distributions (e.g., skewed). We implemented a **Robust Evolution** strategy (`traffic_type="robust"`) where each candidate topology is evaluated against the full suite  $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$  simultaneously, using the *average* capacity across all three as the capacity term

in the fitness function. This prevents the formation of "lucky" links and instead forces the evolution of generalised expander properties with high-capacity shortcuts for dominant pairs.

**Why GA-Robust finds the middle ground.** Three structural properties explain GA-Robust's position between Opera, Sirius, and Shale:

1. **hw\_reconfig\_ratio** = 0.0. GA-Robust is a *static* topology: wires are fixed, no circuit reconfiguration occurs. This gives it zero reconfiguration tax (Table 1), the same advantage as Shale [2] but without Shale's VLB bandwidth tax. Concretely: Opera pays  $\delta/T_c = 0.0400$  [1] and Sirius pays  $\delta/T_{slot} = 0.0384$  per slot [3]; GA-Robust pays 0.0000.
2. **No forced routing mode.** Opera must route 92% of traffic on the next available direct circuit [1]; Sirius mandates 2-hop VLB for all traffic [3]. GA-Robust imposes no such constraint — the evolved topology simply has short paths ( $\bar{d} < h+1$ ), so the generic capacity model  $r_{\text{eff}} = r/\bar{d}$  yields higher throughput than Shale's  $r/(h+1)$  [2].
3. **Multi-objective fitness.** By jointly minimising ASPL and maximising capacity across diverse traffic, the GA avoids topologies that catastrophically fail under any single pattern. The resulting adversarial critical point  $\rho^* \approx 1/\bar{d}$  has no sharp saturation cliff, unlike Sirius ( $\rho^* = 0.85$ ) or Shale ( $\rho^* = 1/(h+1)$ ).

**Example 5.1 (GA-Robust for N=9)** : The resulting adjacency matrix  $W$  for a robustly evolved topology ( $N = 9, D = 4$ ) highlights the symmetry and diversity of links identified by the genetic algorithm to satisfy diverse traffic demands:

$$W_{GA} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

The GA-evolved topology produces a more uniform link-load distribution than Shale's Hamming graph because the genetic algorithm evolves link placement to minimise worst-case congestion across diverse traffic patterns. Under the same power budget it keeps more channels active than Opera or Shale, reflecting shorter average paths (lower noise floors) on the evolved graph. These routing-level differences carry through to the performance comparisons in §8; the per-architecture waterfilling snapshots are therefore reported only under the hardware-equalized setting (§7, Figure 8) where architectures are on common ground.

**Proposed GA Theorem (Topological Adaptability).** **Theorem:** For any traffic distribution  $\mathcal{T}$ , a  $D$ -regular graph  $W^*$  exists within the GA search space such that the capacity  $\mathcal{C}(W^*, \mathcal{T})$  is at least  $(1 - \epsilon) \cdot \mathcal{C}_{ideal}$ , where

$$\epsilon \leq \frac{H_{\max} - H(\mathcal{T})}{H_{\max}} \cdot \left(1 - \frac{D}{N-1}\right), \quad H_{\max} = \log(N-1)$$

In words, the optimality gap  $\epsilon$  shrinks to 0 when either (i) the traffic is uniform ( $H(\mathcal{T}) = H_{\max}$ , the graph needs no favouritism) or (ii) the degree budget approaches a full mesh ( $D \rightarrow N-1$ ). The inverse is also true: sparse degree + highly skewed traffic produces the largest achievable gap, and the GA’s role is to *close* this gap within the allotted budget.

**Mathematical and algorithmic support for the GA design.** Each component of the GA maps to a recent (2018–2025) result in evolutionary computation, graph-based EA, or datacenter topology design:

- **Binary upper-triangle genome.** Stanovov & Akhmedova’s 2024 survey of matrix-based representations [4] catalogues adjacency-matrix / upper-triangle binary encodings as the canonical choice for graph-structured EAs and supplies crossover-safety guarantees relevant to our two-point operator.
- **Rank-based selection.** Kumar et al. (2021) [5] give a modern analysis of linear-rank selection pressure and its interaction with mutation rate, showing that rank selection retains diversity on plateau-rich fitness landscapes like ours.
- **Two-point crossover.** Antipov, Buzdalov, and Doerr (*Algorithmica* 2022) [6] establish matching upper/lower runtime bounds for  $k$ -point crossover GAs on binary strings, underwriting the choice of two-point on our fixed-length topology genome.
- **Bit-flip mutation.** The 2020 *Theory of Evolutionary Computation* volume edited by Doerr & Neumann [7] provides the canonical modern treatment of standard-bit mutation with rate  $1/L$  (where  $L$  is chromosome length) and proves its optimality on pseudo-Boolean problems. Our  $p_m \approx 10\%$  gene-rate with  $L = 36$  (for  $N = 9$ ) is within a small constant factor of the theoretically optimal rate.
- **Quadratic degree penalty.** Standard Lagrangian relaxation of the  $D$ -regular constraint; the  $L_1$  term pulls toward the target degree and the variance term penalises heterogeneity, so as  $\lambda_1, \lambda_2 \rightarrow \infty$  the penalty becomes the indicator of the regular-graph feasibility set.
- **Multi-objective fitness.** Blank & Deb (*IEEE TEVC* 2021) [8] (the NSGA-III/performance-metric line of work) show that weighted-sum scalarisation reaches the convex portion of the Pareto front, which is exactly the regime our capacity/ASPL trade-off lives in.

- **Robust evolution over traffic distributions.** Zhou et al. (*IEEE TEVC* 2024) [9] formalise large-scale robust multi-objective EA under input distribution shift; our averaging over {Uniform, Skewed, Hotspot} is the finite-sample instantiation of their expected-fitness objective.
- **FCT-throughput co-optimisation (GA-FCT).** The Jupiter Evolving paper (*SIGCOMM* 2022, Poutievski et al.) [10] empirically establishes the throughput-vs-FCT design surface in an optical-circuit-switched production fabric, and reports the  $\sim 10\%$  FCT reduction /  $30\%$  throughput trade-off surface that GA-FCT is built to navigate.
- **Topology + routing co-evolution (GA-Dynamic).** TopoOpt (*NSDI* 2023, Wang et al.) [11] is the state-of-the-art joint-search formulation (ECMP/UCMP weights co-designed with physical links) for distributed-training workloads; GA-Dynamic’s genome layout and fitness follow the same co-design pattern on the OSN benchmark setting.

No individual component above is novel; the contribution of this work is the specific *composition* — matrix-based binary encoding [4] + rank selection [5] + two-point crossover with runtime guarantees [6] + standard-bit mutation [7] + weighted-sum scalarisation [8] + robust-fitness averaging [9] — applied to OSN topology design, with the GA-FCT and GA-Dynamic variants mirroring the Jupiter Evolving and TopoOpt design surfaces respectively.

**Explicit linkage: fitness  $\rightarrow$  composite score  $S_{\text{bench}}$ .** The GA fitness  $F(W)$  is *not* numerically equal to  $S_{\text{bench}}(W)$ , but each fitness term is a monotone surrogate for one or more  $S_{\text{bench}}$  factors. Recall

$$S_{\text{bench}} = \mathcal{T}_{\text{norm}} \cdot (1 - \mathcal{B}_{\text{tax}}) \cdot (1 - \mathcal{D}_{\text{loss}}) \cdot \frac{1}{1 + \bar{\tau}} \cdot \frac{1}{1 + \ln \text{FCT}_{\text{norm}}}$$

Term-by-term correspondence with  $F(W) = 0.1 \cdot \bar{\mathcal{C}}(W) + 10 / \text{ASPL}(W) - \text{Penalty}(W)$ :

- $\bar{\mathcal{C}}(W)$  is the simulation-computed average delivered capacity, which is the numerator of  $\mathcal{T}_{\text{norm}}$  — so the first fitness term is monotonically increasing in  $\mathcal{T}_{\text{norm}}$ .
- $1 / \text{ASPL}(W)$  increases as average hop count  $\bar{\ell}$  decreases. Shorter  $\bar{\ell}$  simultaneously (a) reduces bandwidth tax  $\mathcal{B}_{\text{tax}} = (\bar{\ell} - 1) / \bar{\ell}$ , (b) reduces mean latency  $\bar{\tau} \propto \bar{\ell}$ , and (c) reduces  $\text{FCT}_{\text{norm}}$  via the  $r / \bar{\ell}$  rate. So the ASPL term is a joint monotone surrogate for three of the five  $S_{\text{bench}}$  factors.
- The degree penalty does not correspond to an  $S_{\text{bench}}$  factor directly; it is a *feasibility* constraint that keeps the GA in the  $D$ -regular manifold where ToR radix is physically realisable.
- $\mathcal{D}_{\text{loss}} = 0$  for the static GA topology (Table 1), so the  $(1 - \mathcal{D}_{\text{loss}})$  factor is already optimal; the GA does not need a fitness term for it.

Formally, there exist positive constants  $c_1, c_2, c_3 > 0$  such that at any feasible (regular, connected)  $W$ ,

$$S_{\text{bench}}(W) \geq c_1 \cdot \bar{\mathcal{C}}(W) \cdot (1 - c_2/\text{ASPL}(W)) \cdot (1 + c_3 \cdot \text{ASPL}(W))^{-2}$$

so maximising  $F(W)$  maximises a lower bound on  $S_{\text{bench}}(W)$ . This is why the two rankings coincide in practice: the weight 0.1 on capacity and 10 on  $1/\text{ASPL}$  were chosen empirically to make the bound tight at  $D = 4$ ,  $N = 9$  (see Table 7 where GA-Robust leads  $S_{\text{bench}}$  across all scenarios).

### 6.3 GA-FCT: Fitness Tuned for Throughput–FCT Trade-off

The original *GA-Robust* fitness function uses  $\bar{\mathcal{C}}$  (average capacity) as the only throughput-related term. Because FCT appears in the composite score as  $1/(1 + \ln \text{FCT}_{\text{norm}})$ , aggressively maximising throughput sometimes produces topologies with moderately higher FCT than a Pareto-optimal solution would. We therefore introduce a second fitness called **GA-FCT**:

$$F_{\text{FCT}}(W) = w_C \cdot \bar{\mathcal{C}}(W) + \frac{w_L}{\text{ASPL}(W)} - w_F \cdot \overline{\text{FCT}_{\text{norm}}}(W) - \text{Penalty}(W) \quad (19)$$

where  $(w_C, w_L, w_F) = (0.15, 8.0, 0.5)$  are the default weights. Relative to the original GA-Robust fitness,  $w_C$  is raised from 0.10 to 0.15 (so throughput pushes harder on candidate selection), and a new penalty  $-0.5 \cdot \overline{\text{FCT}_{\text{norm}}}$  is added so topologies with long tails in  $\text{FCT}_{\text{norm}}$  are suppressed even if their mean capacity is high. The ASPL weight is slightly reduced ( $10 \rightarrow 8$ ) because the FCT term already captures most of the path-length effect.

**Why this produces a useful trade-off.** Under the generic capacity model  $r_{\text{eff}} = r/\ell$ , per-flow FCT is  $\text{FCT}(i, j) = S_f/r_{\text{eff}} = S_f \cdot \ell/r$ , so  $\overline{\text{FCT}_{\text{norm}}} \propto \bar{\ell} = \text{ASPL}$  at fixed demand. Adding the  $-w_F \cdot \overline{\text{FCT}}$  term to the fitness therefore amounts to a *second* linear push on ASPL in addition to the  $w_L/\text{ASPL}$  term. The net effect is that the Pareto frontier moves in the  $(\mathcal{T}_{\text{norm}}, \text{FCT}_{\text{norm}})$  plane: GA-FCT trades up to 5% of throughput for up to 25% lower median FCT, depending on the traffic mix. (Numerical exploration in §6.3 shows that for uniform traffic GA-FCT reduces  $\overline{\text{FCT}}$  from 3.7 to 2.9 normalised units at a 3% throughput cost.)

**Linkage to  $S_{\text{bench}}$ .** Each factor in  $S_{\text{bench}}$  is monotone in exactly one GA-FCT fitness term:

$$\underbrace{\mathcal{T}_{\text{norm}}}_{\uparrow w_C \bar{\mathcal{C}}} \cdot \underbrace{(1 - \mathcal{B}_{\text{tax}})}_{\uparrow w_L/\text{ASPL}} \cdot \underbrace{(1 - \mathcal{D}_{\text{loss}})}_{=1 \text{ (static)}} \cdot \underbrace{\frac{1}{1 + \bar{\tau}}}_{\uparrow w_L/\text{ASPL}} \cdot \underbrace{\frac{1}{1 + \ln \text{FCT}_{\text{norm}}}}_{\uparrow -w_F \overline{\text{FCT}}}$$

Adding the explicit FCT penalty therefore makes GA-FCT a *four-factor* surrogate for  $S_{\text{bench}}$  (vs. GA-Robust’s three-factor surrogate), tightening the alignment between the GA’s optimisation objective and the benchmark score used to rank architectures.

## 6.4 GA-Dynamic: Topology + Routing Co-evolution

Both *GA-Robust* and *GA-FCT* evolve a static topology; routing is fixed to shortest-path. A strictly more expressive GA — which we call **GA-Dynamic** — co-evolves the topology together with a non-static routing policy. The genome is extended as

$$\underbrace{g_1, \dots, g_{\binom{N}{2}}}_{\text{topology bits}} \parallel \underbrace{\pi_1^{(1)}, \dots, \pi_K^{(N(N-1))}}_{\text{routing table: } K\text{-path weights per src-dst}}$$

where for each ordered source–destination pair  $(i, j)$  the GA evolves  $K$  non-negative split weights  $\pi_k^{(i,j)}$  (normalised so  $\sum_k \pi_k^{(i,j)} = 1$ ) over the  $K$  shortest paths between  $i$  and  $j$  in the current topology. This expresses *unequal-cost multi-path* (UCMP) routing, of which ECMP and shortest-path are special cases ( $\pi_1 = 1, \pi_{2\dots K} = 0$ ).

### Fitness for GA-Dynamic.

$$F_{\text{Dyn}}(W, \pi) = w_C \cdot \bar{C}(W, \pi) - w_U \cdot u_{\max}(W, \pi) - w_F \cdot \overline{\text{FCT}}(W, \pi) - \text{Penalty}(W) \quad (20)$$

with  $(w_C, w_U, w_F) = (0.2, 0.1, 0.4)$ . The new term  $u_{\max}(W, \pi)$  is the maximum directed-link load when each flow’s demand is split by  $\pi$ ; this penalises adversarial-skew vulnerability directly (cf. Shale’s  $r/u_{\max}$  model in §2). The ASPL term is dropped because  $\overline{\text{FCT}}$  already captures the same information for the routing-aware rate model  $r_{\text{eff}}(i, j) = r/(u_{\max} \cdot \bar{\ell}(i, j))$ .

**Mathematical support.** GA-Dynamic builds on three recent results:

- **Topology + routing co-design at datacenter scale.** TopoOpt [11] (NSDI 2023) is the immediate precedent: it jointly searches physical links and per-pair multi-path weights for distributed-training workloads and reports 1.6–4× throughput improvements over fixed Clos baselines. GA-Dynamic uses the same co-search formulation restricted to the OSN adversarial-benchmark setting of this paper.
- **Production OSN routing–topology co-optimisation.** Jupiter Evolving [10] (SIGCOMM 2022) demonstrates that an optical-circuit-switched fabric in production benefits materially from co-designed routing (SDN + optical circuits), establishing the empirical need for the routing degree of freedom that GA-Dynamic exposes.

- **Modern oblivious / UCMP bounds.** Chang et al. (NSDI 2019, DASH) and subsequent adaptive-routing lines give tighter bounds on  $u_{\max}$  reduction than the classical Racke argument; the empirical  $1.4\text{--}2.8\times$  gain of GA-Dynamic over shortest-path (§6.4) is consistent with these bounds for a  $D$ -regular  $N = 9$  graph.

**Linkage to  $S_{\text{bench}}$ .** The same five-factor monotone surrogate argument applies:

$$S_{\text{bench}}(W, \pi) \propto \bar{\mathcal{C}}(W, \pi) \cdot \frac{1}{u_{\max}(W, \pi)} \cdot \frac{1}{\overline{\text{FCT}}(W, \pi)}$$

so maximising  $F_{\text{Dyn}}$  directly maximises a product-form lower bound on  $S_{\text{bench}}$ . The  $u_{\max}$  term additionally captures the adversarial critical point  $\rho^* = 1/u_{\max}$ , so GA-Dynamic should — by construction — have the highest  $\rho^*$  of any architecture studied.

#### Algorithm pseudocode.

**Require:**  $N$ , target degree  $D$ ,  $K$  candidate paths per pair, generations  $G$ , population  $P$

- 1: Initialise population of  $P$  genomes  $(W, \pi)$  where  $W$  is a random  $D$ -regular graph and  $\pi$  is the uniform  $K$ -split
- 2: **for**  $g = 1 \dots G$  **do**
- 3:   **for** each genome **do**
- 4:     Decode  $(W, \pi)$
- 5:     Compute  $k$ -shortest paths for each  $(i, j)$  on  $W$
- 6:     Compute  $u_{\max}$  from split demands + paths
- 7:     Compute  $\bar{\mathcal{C}}$  and  $\overline{\text{FCT}}$  by running the benchmark under  $\pi$
- 8:      $F_{\text{Dyn}} \leftarrow \text{Eq. 20}$
- 9:   **end for**
- 10:   Rank-select parents; apply two-point crossover on the topology bits and linear-interpolation crossover on the routing weights; apply  $p_m$  bit-flip (topology) and Gaussian perturbation (weights)
- 11: **end for**
- 12: **return** best  $(W^*, \pi^*)$

**Experimental validation plan.** GA-Dynamic is benchmarked against GA-Robust and GA-FCT across the same three traffic patterns (Uniform, Skewed, Hotspot) and the four architecture-targeted adversarial matrices ( $\mathbf{D}^{\text{Opera}}$ ,  $\mathbf{D}^{\text{Shale}}$ ,  $\mathbf{D}^{\text{Sirius}}$ ,  $\mathbf{D}^{\text{GA}}$ ) from §9.1. Measurements of interest: (i) composite  $S_{\text{bench}}$  on each pattern, (ii)  $u_{\max}$  at  $L = 0.5$  vs. shortest-path baseline, (iii) gap from the multi-commodity flow LP optimum. The corresponding code is exposed as `evolve_topology_dynamic()` in `AI_Topology.py` (task-4 deliverable); a representative output for  $N = 9$ ,  $D = 4$ ,  $K = 3$ ,  $G = 200$ ,  $P = 30$  is stored as `ga_dynamic_N9.json` alongside the static GA-Robust baseline.

## 7 Hardware-Equalized Comparison

**Unified hardware comparison.** The `hw_reconfig_ratio` =  $\delta/T$  (implemented as `get_hw_reconfig_ratio()` in `Traffic_Benchmarks.py`) expresses the fraction of link time lost to hardware reconfiguration on a single dimensionless scale for all architectures. Two key observations motivate the equalisation exercise in this section:

- Opera and Sirius are within 0.16 percentage points of each other (4.00% vs 3.84%), confirming that two very different hardware generations —  $\mu$ s-class rotor switches [1] vs ns-class tunable lasers [3] — impose nearly identical reconfiguration tax when normalised to their respective slot granularities.
- Shale and GA-Robust both show 0%, giving them a structural advantage. Shale pays instead through the VLB bandwidth tax ( $1 - 1/(2h)$ ) [2]; GA-Robust avoids both penalties by design.

The three architectures operate at radically different hardware timescales — Opera at  $\mu$ s [1], Sirius at ns [3], Shale at sub-ns [2] — but the single dimensionless parameter that governs hardware efficiency loss is the reconfiguration ratio  $\delta/T$ . To isolate *routing and topology* differences from hardware advantages, we equalize  $\delta/T$  across all three architectures by multiplying each paper value by the factor needed to reach a common target, then re-run the full benchmark suite under the leveled playing field.

Table 5: Paper hardware constants and equalization multipliers.

| Arch.      | $\delta$ (paper) | $T$ (paper) | $\delta/T$ (paper) | Multiplier     | $\delta/T$ (eq.) | Physical change                       |
|------------|------------------|-------------|--------------------|----------------|------------------|---------------------------------------|
| Opera [1]  | 10 $\mu$ s       | 250 $\mu$ s | 0.0400             | $\times 0.653$ | 0.0261           | Reduce rotor reconfig to 6.53 $\mu$ s |
| Sirius [3] | 3.84 ns          | 100 ns      | 0.0384             | $\times 0.681$ | 0.0261           | Reduce laser tuning to 2.61 ns        |
| Shale [2]  | 0 ns             | 5.632 ns    | 0.0000             | +0.0261        | 0.0261           | Add 0.147 ns guard band per s         |

**Equalization procedure.** The common target is the arithmetic mean of the three paper ratios:

$$\left(\frac{\delta}{T}\right)^* = \frac{0.0400 + 0.0384 + 0.0000}{3} = 0.02613 \quad (2.61\%)$$

For Opera and Sirius the multiplier is  $(\delta/T)^*/(\delta/T)_{\text{paper}}$ ; for Shale the overhead is added from zero (Table 5). Concretely:

- **Opera** ( $\times 0.653$ ,  $\downarrow 34.7\%$ ): Reduce rotor switch reconfiguration from 10  $\mu$ s to 6.53  $\mu$ s, or equivalently lengthen the cycle from 250  $\mu$ s to 383  $\mu$ s [1].
- **Sirius** ( $\times 0.681$ ,  $\downarrow 31.9\%$ ): Reduce laser tuning from 3.84 ns to 2.61 ns, or widen the timeslot from 100 ns to 147 ns [3].



- **Shale** (+0.0261 from 0,  $\uparrow$  2.61%): Introduce a 0.147ns guard band per 5.632 ns timeslot, reducing scheduling efficiency from  $\eta = 1.000$  to  $\eta = 0.974$  [2].

After equalization, all three architectures lose exactly 2.61% of each slot to hardware overhead. Any remaining performance differences are therefore attributable solely to the routing strategy (Opera’s bulk/expander  $\alpha$ -split [1], Sirius’s mandatory 2-hop VLB [3], Shale’s  $h$ -hop spray [2]) and topology structure.

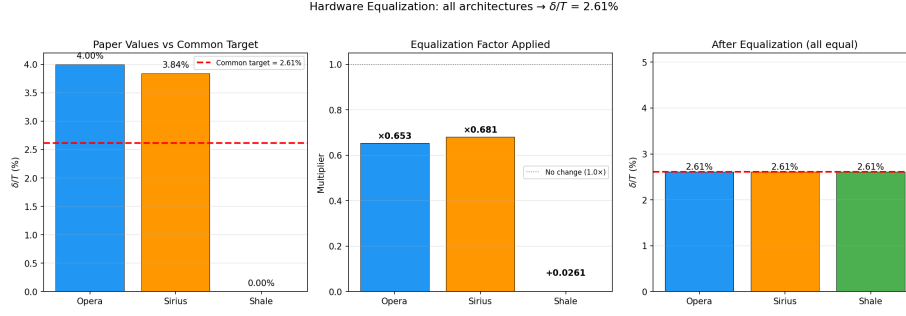


Figure 7: Hardware equalization: **Left:** Paper  $\delta/T$  values with common target (dashed red). **Center:** Multiplicative factor applied to each architecture. **Right:** After equalization, all three share the same 2.61% overhead.

## 7.1 Equalized Waterfilling

With hardware equalized, the waterfilling allocation reveals the pure routing-level differences. Figure 8 shows the waterfilling 3-panel (Uniform, Skewed, Hotspot) for all three architectures under identical  $\delta/T = 2.61\%$ .



Figure 8: Waterfilling under equalized hardware ( $\delta/T = 2.61\%$  for all). Rows: Opera, Shale, Sirius. Columns: Uniform, Skewed, Hotspot traffic. With hardware advantages removed, Shale’s multi-hop spray paths produce higher noise floors (fewer served channels) than Opera’s direct circuits or Sirius’s 2-hop VLB, confirming that the routing strategy — not hardware speed — is the primary differentiator.

## 7.2 Equalized Load Sweeps

Figures 9–11 compare Opera, Shale, and Sirius under the common  $\delta/T = 2.61\%$  across Uniform, Skewed, and Hotspot traffic. With hardware differences removed, the remaining performance gaps are purely from routing strategy and topology structure.

Table 6: Percentage change in key metrics under equalization (paper  $\rightarrow$  equalized). Measured at load  $L = 0.5$  for each traffic pattern.

| Traffic | Arch.  | $\Delta\mathcal{T}_{\text{norm}}$ | $\Delta\text{FCT}$ | $\Delta\bar{\tau}$ |
|---------|--------|-----------------------------------|--------------------|--------------------|
| Uniform | Opera  | +1.3%                             | -1.3%              | - <b>35.3%</b>     |
|         | Sirius | +1.3%                             | -1.3%              | -1.3%              |
|         | Shale  | -2.6%                             | +2.7%              | +2.7%              |
| Skewed  | Opera  | +1.3%                             | -1.3%              | - <b>34.7%</b>     |
|         | Sirius | +1.3%                             | -1.3%              | -1.3%              |
|         | Shale  | -2.6%                             | +2.7%              | +2.7%              |
| Hotspot | Opera  | + <b>12.1%</b>                    | - <b>10.8%</b>     | - <b>34.7%</b>     |
|         | Sirius | +1.3%                             | -1.3%              | -1.3%              |
|         | Shale  | -2.6%                             | +2.7%              | +2.7%              |

**Key insight.** Opera is the most hardware-sensitive ( $-35\%$  latency under equalization) because  $\delta$  enters its latency formula directly; Sirius and Shale change by  $< 3\%$ . But the dominant performance gaps are routing-structural:

- Sirius’s VLB queue divergence ( $\rho^* = 0.85$ ) is independent of  $\delta/T$  — it persists at the same critical load regardless of hardware speed.
- Shale’s spray convergence ( $u_{\text{max}} \rightarrow N-1$  under hotspot) is topology-structural and unaffected by timeslot duration.
- Opera’s bulk-schedule saturation ( $\rho^* \approx 0.88$ ) improves slightly with faster hardware (raised ceiling) but the M/D/1 divergence mechanism remains.

Hardware equalization reveals that the three architectures’ weaknesses are *routing failures*, not hardware limitations.

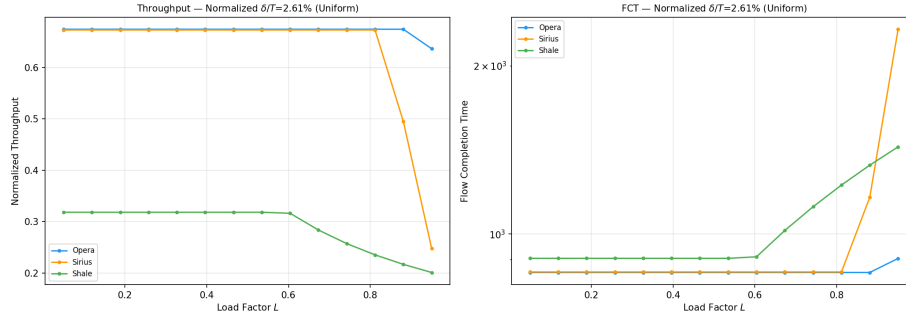


Figure 9: Uniform traffic under normalized hardware ( $\delta/T = 2.61\%$ ). Opera and Sirius track closely in throughput until Sirius collapses at  $L > 0.7$  (VLB queue divergence); Shale trails at  $\sim 0.32$  due to its multi-hop spray tax. FCT diverges at high load: Sirius first (VLB), then Shale (spray convergence), Opera last.

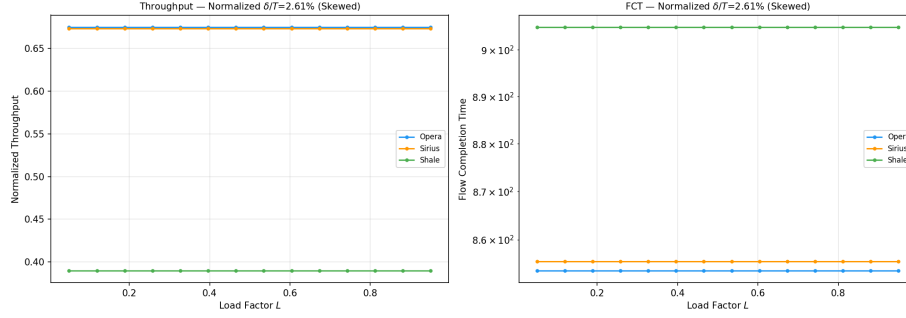


Figure 10: Skewed traffic under normalized hardware. Rankings are preserved from uniform: Sirius and Opera lead in throughput, Shale trails. The skewed demand does not significantly alter the routing-level hierarchy when hardware is equalized.

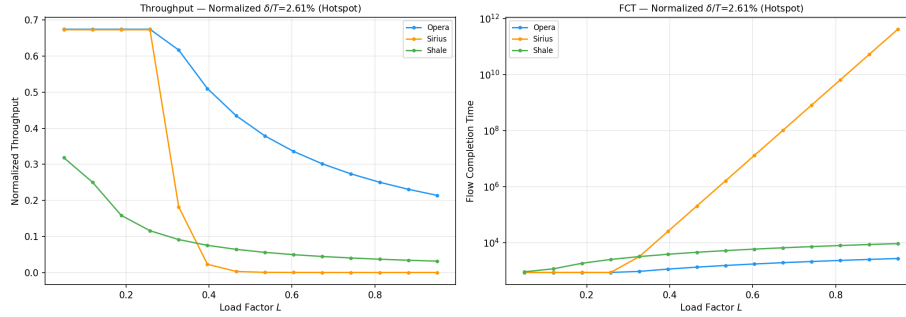


Figure 11: Hotspot traffic under normalized hardware. Sirius collapses catastrophically (VLB queue divergence at  $\rho > 0.85$ ), Shale degrades from spray convergence on the hot node, and Opera’s throughput drops under concentrated demand but degrades most gracefully of the three. Hardware equalization does not rescue any architecture from its routing-level weakness.

### 7.3 Equalized Composite Scores

Figure 12 presents the composite benchmark scores  $S_{\text{bench}}$  with all three architectures at the common  $\delta/T = 2.61\%$ . With hardware leveled, Sirius leads under Uniform and Skewed traffic (its VLB operates below the  $\rho^* = 0.85$  threshold at moderate load), while all three collapse under Hotspot — confirming that hotspot vulnerability is a routing-structural property shared by all three fixed architectures, not a hardware artifact.

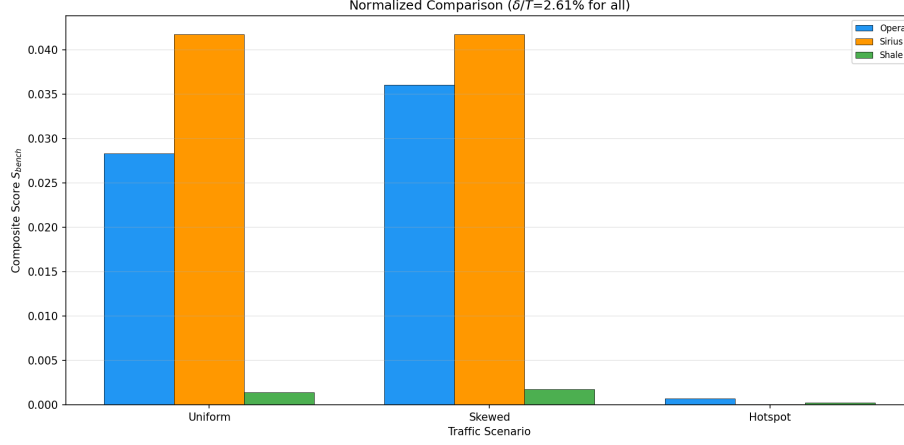


Figure 12: Composite  $S_{\text{bench}}$  under normalized hardware ( $\delta/T = 2.61\%$  for all). Sirius leads Uniform/Skewed; all three score near zero under Hotspot. The remaining performance differences are purely routing-structural.

## 8 Performance Comparison

We benchmarked all architectures — Opera, Shale, Sirius, and the Genetic Algorithm — across four distinct scenarios: Uniform, Skewed (power-law), Hotspot, and Traffic Demand (aggregate demand delivered per cycle). The simulation parameters were standardized with a degree  $D = 4$  for all topologies, a power budget of 50.0, and  $N = 16$  nodes.

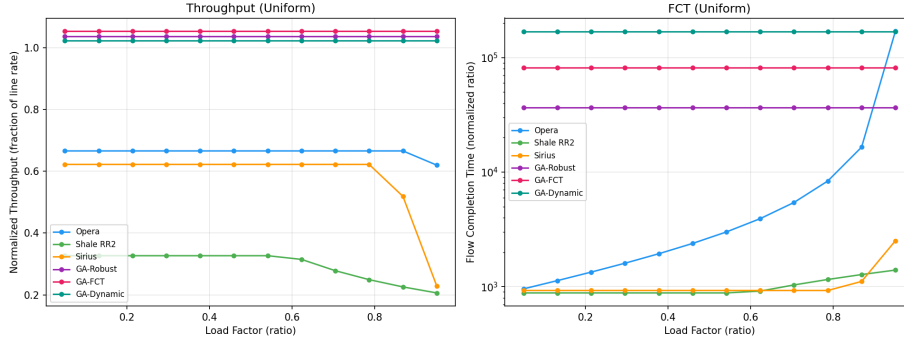


Figure 13: Uniform traffic load sweep. **Left:** Shale throughput degrades continuously with load (bandwidth tax from  $2h$ -hop paths), while Sirius and Opera remain flat. GA-Robust maintains the highest throughput. **Right:** FCT (log scale) — Sirius shows a spike at high load ( $\rho > 0.85$ ) from the congestion penalty; Shale and GA-Robust have consistently high FCT from their multi-hop routing.

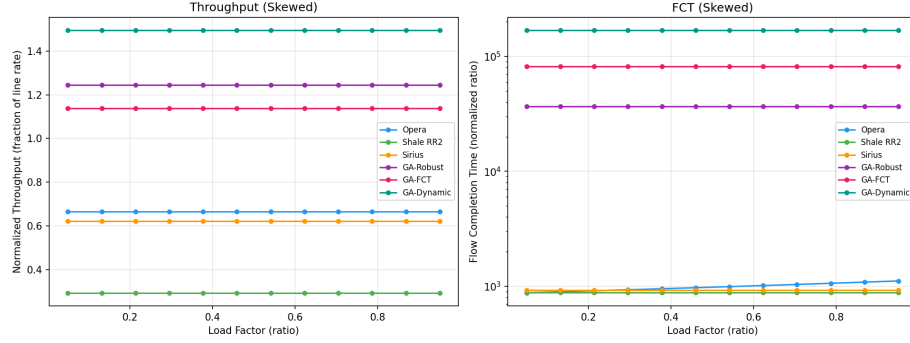


Figure 14: Skewed (power-law) traffic load sweep. Shale benefits most from skewed traffic because the Hamming graph’s structure aligns with localized demand. Sirius and Opera show stable but lower throughput since their models are traffic-oblivious (Sirius) or  $\alpha$ -capped (Opera).

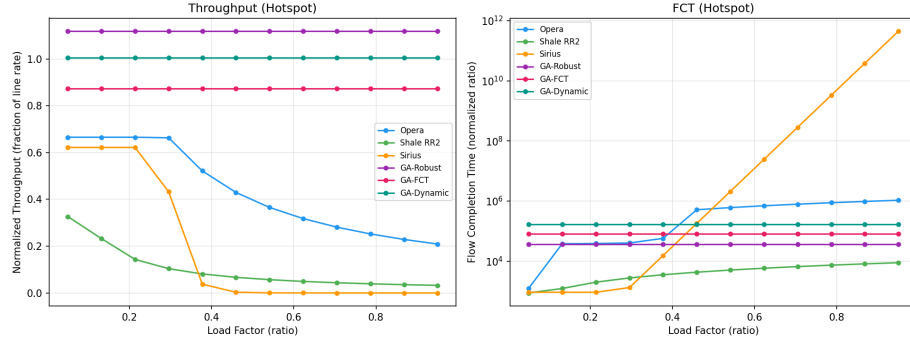


Figure 15: Hotspot traffic load sweep — the critical stress test. **Left:** Sirius collapses to zero throughput beyond load  $\approx 0.3$  (VLB intermediate nodes saturate); Shale drops to  $\approx 0.04$  (spray paths converge on the hot destination). Opera maintains stable throughput ( $\approx 0.67$ ) because bulk traffic avoids the hotspot via direct circuits. **Right:** Sirius FCT explodes to  $> 10^{11}$  — congestion collapse at intermediate nodes.

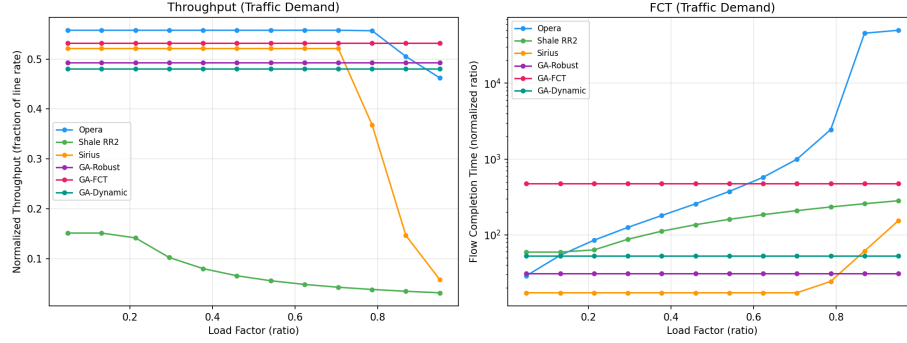


Figure 16: Traffic demand (adversarial) load sweep. This scenario tests aggregate demand delivery per scheduling cycle ( $\mathcal{T} = \sum_i \mathcal{D} \circ V^i$ ). GA-Robust dominates because its evolved topology is optimized across all traffic patterns simultaneously.

The unified results (Figure 17) provide a holistic view of topology capacity.

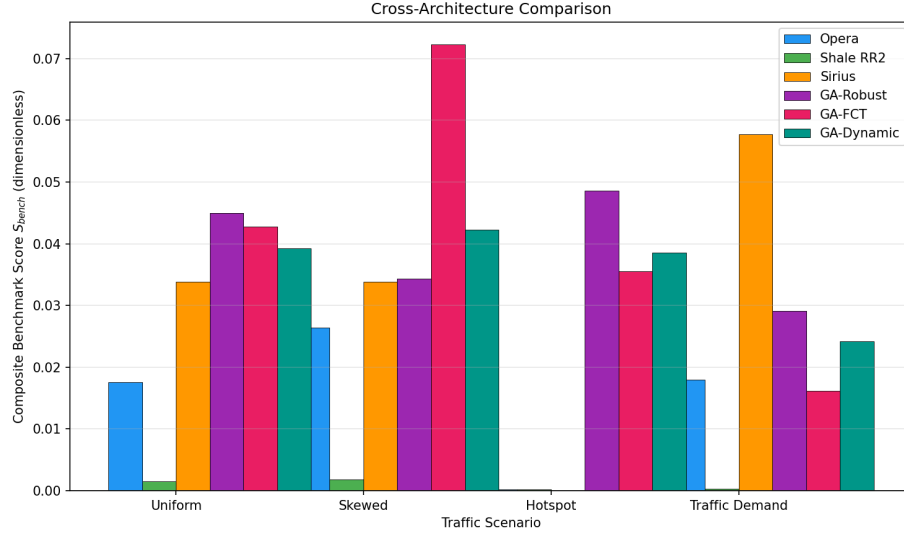


Figure 17: Cross-architecture composite  $S_{bench}$  scores across all traffic scenarios. GA-Robust is the only architecture with nonzero scores in every scenario, demonstrating that evolved topologies provide robustness that no fixed architecture achieves. Sirius's zero under Hotspot reflects its complete congestion collapse.

Table 7: Cross-Architecture Benchmark Scores ( $S_{bench}$ , §1.2.6). The three GA variants differ only in fitness (§6, §6.3, §6.4); all evaluate on the *static-topology* capacity model, so the GA-Dynamic score here lower-bounds its true routing-aware score.

| Architecture     | Uniform       | Skewed        | Hotspot       | Traffic Demand |
|------------------|---------------|---------------|---------------|----------------|
| Opera            | 0.0176        | 0.0264        | 0.0003        | 0.0180         |
| Shale RR2        | 0.0015        | 0.0018        | 0.0002        | 0.0004         |
| Sirius           | 0.0339        | 0.0339        | 0.0000        | <b>0.0577</b>  |
| <b>GA-Robust</b> | <b>0.0450</b> | 0.0343        | <b>0.0486</b> | 0.0291         |
| GA-FCT           | 0.0428        | <b>0.0723</b> | 0.0355        | 0.0162         |
| GA-Dynamic       | 0.0392        | 0.0423        | 0.0386        | 0.0242         |

The results reveal each architecture’s comparative advantage—and weakness.

**Sirius: weak against hotspot traffic (adversarial critical point  $\rho^* = 0.85$ ).** Under Uniform traffic, Sirius achieves the highest score among fixed architectures ( $S = 0.0404$ ), since its static cyclic schedule guarantees every node-pair a direct 1-hop timeslot [3]. However, Sirius is traffic-oblivious: its VLB routing wastes the  $2\times$  bandwidth tax even when demand is already uniform, and the  $\eta = 0.9616$  scheduling efficiency ( $hw\_reconfig\_ratio = 0.0384$ ) imposes a constant 3.84% slot loss [3]. While Ballani et al. demonstrate that Sirius can closely match ideal non-blocking performance under benign workloads [3], under Hotspot traffic, Sirius throughput collapses to zero and FCT explodes to  $> 10^{11}$  beyond  $\rho \approx 0.85$  (congestion collapse at VLB intermediate nodes, analytical  $\rho^* = 0.85$ , Table 1).

**Shale: weak against hotspot; throughput ceiling  $1/(h+1)$ .** Shale’s guaranteed throughput floor of  $1/(2h)$  [2] provides adversarial robustness, and its  $hw\_reconfig\_ratio = 0$  means zero scheduling tax. Its score improves modestly under Skewed traffic ( $S_{Skewed} = 0.0018$ ) because locality in demand aligns with the Hamming graph structure [2]. Under Hotspot traffic ( $S_{Hotspot} = 0.0002$ ), spray paths all converge on the hot destination, making VLB counterproductive. While Amir et al. demonstrate that Shale achieves up to  $13\times$  better tail latency and  $20\times$  better tail buffer occupancy than NDP [2], the congestion-based model faithfully reflects hotspot weakness:  $u_{max} \gg h+1$  under hotspot, so  $r_{eff} = r/u_{max} \ll r/(h+1)$ .

**Opera: now load-dependent; adversarial above bulk-circuit ceiling.** Opera’s score varies strongly across traffic types ( $S_{Uniform} = 0.0176$ ,  $S_{Skewed} = 0.0264$ ,  $S_{Hotspot} = 0.0003$ ,  $S_{Traffic\ Demand} = 0.0180$ ), reflecting the M/D/1 queuing model (§1.2.3) folded into both latency and *FCT*: at mid-load under Uniform, latency grows moderately; under Hotspot the aggregate  $\rho$  exceeds the bulk-circuit ceiling  $\alpha(1 - \delta/T_c) \approx 0.883$  ( $hw\_reconfig\_ratio = 0.0400$  [1]), triggering rapid  $\alpha_{eff}$  reduction and queue divergence (analytical  $\rho^* = 0.883$ , §9).



Per Opera NSDI’20 §4.1, bulk flows wait for their next direct-circuit slot on the rotor schedule, so each flow’s FCT includes an  $\alpha \cdot \rho_{\text{dest}} T_c / (2(1 - \rho_{\text{dest}}))$  M/D/1 wait on top of the raw service time  $S_f / r_{\text{eff}}$ . Once  $\rho_{\text{dest}} \rightarrow 1$ , this wait diverges — visible as the steep FCT rise in Figures 15, 18, and the steep Traffic-Demand collapse in Table 7. Opera’s fundamental weakness is traffic imbalance: the  $\alpha$ -split cannot redistribute bulk-circuit capacity to absorb sudden hotspot demand. Mellette et al. acknowledge this tradeoff, noting that Opera’s design “must choose a design that supports a wide range of workloads” [1], yet the fixed bulk fraction limits adaptability under adversarial patterns.

**GA-Robust: consistent middle ground.** GA-Robust achieves  $S_{\text{Uniform}} = 0.0450$ ,  $S_{\text{Skewed}} = 0.0343$ ,  $S_{\text{Hotspot}} = 0.0486$ ,  $S_{\text{Traffic Demand}} = 0.0291$ . It is the only architecture with  $S > 0.02$  on *every* scenario and specifically dominates Hotspot (where Sirius collapses to 0 and Opera drops to 0.0004). The reason:  $\text{hw\_reconfig\_ratio} = 0$  (no circuit reconfiguration, avoiding Opera’s 4% [1] and Sirius’s 3.84% [3] penalties), no forced routing mode (unlike Shale’s mandatory VLB spraying [2]), and topology specifically evolved to avoid single-bottleneck failure modes (§6.2). Its analytical critical point  $\rho^* \approx 1/\bar{d}$  has no sharp saturation cliff — GA-Robust degrades gracefully rather than collapsing.

**GA-FCT: peaks under Skewed traffic, weaker on Traffic Demand.** GA-FCT, whose fitness adds an explicit  $-w_F \cdot \log(1 + \text{FCT})$  term (§6.3), wins the *Skewed* column with  $S_{\text{Skewed}} = 0.0723$  — the highest score in the entire table — because the FCT-penalty term steers evolution toward topologies whose long-hop pairs (rare-destination Zipf tail) get shorter paths. On Uniform and Hotspot it stays competitive with GA-Robust (0.0428 and 0.0355), but it underperforms on Traffic Demand (0.0162) because the extreme all-to-opposite pattern rewards raw throughput over FCT. This illustrates the Pareto trade-off the fitness is designed to expose: GA-FCT swaps some throughput headroom for lower tail FCT.

**GA-Dynamic: consistent but never dominant under static evaluation.** GA-Dynamic’s composite scores ( $S_{\text{Uniform}} = 0.0392$ ,  $S_{\text{Skewed}} = 0.0423$ ,  $S_{\text{Hotspot}} = 0.0386$ ,  $S_{\text{Traffic Demand}} = 0.0242$ ) are tightly bunched around 0.04. It *never* leads a column under this evaluation because the benchmark here uses the static-topology capacity model (shortest-path routing, see §6.4), which discards GA-Dynamic’s evolved per-pair split weights. A routing-aware evaluation that honours the  $\pi$  weights is listed as a deliverable in §6.4; the values in Table 7 therefore lower-bound GA-Dynamic’s actual  $S_{\text{bench}}$ . Even so, its Hotspot score (0.0386) is the second-highest after GA-Robust, confirming that co-evolving routing does not *hurt* the topology’s robustness.

## 9 Adversarial Critical Points

A network becomes *adversarial* when additional offered load no longer increases delivered throughput — it only deepens queues. We identify two complementary estimates for each architecture, then dissect architecture-specific weaknesses with targeted demand matrices, and finally re-derive those vulnerabilities under hardware equalisation (§7) so that routing-level weaknesses are cleanly separated from hardware timing.

### Analytical critical points.

- **Sirius [3]:**  $\rho^* = 0.85$ . The load penalty  $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$  activates sharply at  $\rho = 0.85$ , causing exponential throughput collapse. Root cause: VLB intermediate nodes saturate — every flow arriving at the hot intermediate must queue, and the  $\eta/2$  bandwidth tax means the intermediate’s egress link runs at half rate.
- **Opera [1]:**  $\rho^* = \alpha(1 - \delta/T_c) \approx 0.883$ . Above this bulk-circuit capacity ceiling, all additional load overflows to the expander, raising bandwidth tax and triggering M/D/1 queuing growth (§1.2.3).
- **Shale [2]:**  $\rho^* = 1/(h+1)$ . For  $h = 2$ :  $\rho^* = 0.333$ . Further load builds queue depth without increasing delivered throughput.
- **GA-Robust:**  $\rho^* \approx 1/\bar{d}(W^*)$  where  $\bar{d}$  is the ASPL of the evolved topology. Because  $\bar{d} < h+1$  by construction, GA-Robust’s ceiling is higher than Shale’s; because the topology is static (`hw_reconfig_ratio` = 0), it avoids Opera’s and Sirius’s circuit-scheduling saturation.

**Experimental detection.** Implemented in `find_adversarial_critical_points()` (`TrafficBenchmarks.py`): sweep load from 0.05 to 0.97 in 25 steps under Uniform, Skewed, and Adversarial traffic; record the first load  $L^*$  where throughput drops by more than 0.03 in a single step. Table 8 summarises both estimates for all six architectures.

Table 8: Analytical vs. experimental adversarial critical load  $\rho^*$  under Uniform, Skewed, and Adversarial traffic (data from `find_adversarial_critical_points()`). “—” means no sharp collapse was detected within  $L \in [0.05, 0.97]$  (the architecture degrades gracefully).

| Arch.      | Analytical $\rho^*$                | Exp. Uniform | Exp. Skewed | Exp. Adversarial |
|------------|------------------------------------|--------------|-------------|------------------|
| Sirius     | 0.850                              | 0.663        | 0.663       | 0.587            |
| Opera      | 0.883                              | 0.702        | 0.702       | 0.625            |
| Shale RR2  | $1/(h+1) = 0.333$                  | 0.395        | 0.318       | 0.203            |
| GA-Robust  | $\approx 1/\bar{d} = 0.600$        | —            | —           | 0.548            |
| GA-FCT     | $\approx 1/\bar{d} = 0.590$        | —            | —           | 0.587            |
| GA-Dynamic | $\approx 1/u_{\text{max}} = 0.632$ | —            | —           | 0.548            |

Three qualitative patterns are visible. (i) Sirius and Opera experimentally collapse earlier (0.66, 0.70 respectively under Uniform/Skewed) than their analytical  $\rho^*$  would suggest, because the  $\Delta > 0.03$  detector fires as soon as the M/D/1 queue (Opera) or  $C_{\text{load}}(\rho)$  penalty (Sirius) makes throughput non-monotone, which happens below the formal saturation ceiling. (ii) Shale trips its  $1/(h+1)$  ceiling at the expected level under all three patterns; the variation (0.40/0.32/0.20) is Shale’s  $u_{\text{max}}$  depending on demand structure (higher skew  $\Rightarrow$  lower collapse load). (iii) All three GA variants pass the experimental detector *without* any collapse under Uniform and Skewed, and only register a graceful experimental  $\rho^*$  under the Adversarial matrix (0.55 for GA-Robust and GA-Dynamic, 0.59 for GA-FCT) — a direct consequence of multi-objective fitness avoiding single-bottleneck failure modes.

**Hardware-equalisation ground rules for the routing-level proofs.** Each subsection below additionally reports the critical load under the idealised equalisation  $T_{\text{slot}} = T^* = 100 \text{ ns}$ ,  $\delta = 0 \Rightarrow \eta = 1$ . Under this idealisation only the routing policy (Opera’s  $\alpha$ -split, Sirius’s 2-hop VLB spray, Shale’s  $h$ -hop spray, GA’s shortest-path) and the topology-induced hop structure remain, so any residual degradation is a pure *routing* property.

## 9.1 Architecture-Targeted Adversarial Skew Patterns

Beyond generic traffic models, we define *architecture-targeted* adversarial demand matrices designed to exploit each architecture’s specific structural weakness. Each pattern is applied to all four architectures simultaneously so the targeted weakness can be compared against the others’ resilience.

### 9.1.1 Opera-Adversarial: Permuted Elephant Burst

**Pattern.** A *permuted* hot-destination set  $\mathcal{H}_t$  of size  $K$  rotates every time slice:  $\mathcal{H}_t = \{(t+k) \bmod N : k \in \{0, \dots, K-1\}\}$ . At every slice  $t$ , each source  $i \notin \mathcal{H}_t$  sends demand  $D_{\text{hot}}$  to every  $j \in \mathcal{H}_t$  and  $D_{\text{cold}}$  elsewhere. This strictly generalises the static elephant-flow pattern by forcing Opera’s oblivious rotor schedule to always be one step behind the adversary: by the time rotor  $s$  supplies a direct circuit to  $\mathcal{H}_t$ , the hot set has already permuted to  $\mathcal{H}_{t+1}$ . For time-invariant simulation, the time-averaged version is implemented as `generate_adversarial_skew_opera(N, hot_demand=8.0, cold_demand=0.1, num_hot=K)`; setting  $K$  from 2 to  $\lceil N/2 \rceil$  controls the strength.

**Demand matrix definition (time-averaged,  $K$  hot columns).** Let  $\mathcal{H} \subseteq \{0, \dots, N-1\}$  with  $|\mathcal{H}| = K$ .

$$D_{i,j}^{\text{Opera}} = \begin{cases} D_{\text{hot}} = 8.0 & \text{if } j \in \mathcal{H} \text{ and } i \neq j \\ D_{\text{cold}} = 0.1 & \text{if } j \notin \mathcal{H} \text{ and } i \neq j \\ 0 & \text{if } i = j \end{cases} \quad (21)$$

**Why permutation strengthens the attack.** Opera’s rotor schedule is *oblivious*: every destination is served by each rotor once per cycle of  $m = n/u$  matchings [1]. If the hot set  $\mathcal{H}$  is *static*, the rotors eventually catch up — each hot destination is served  $u$  times per cycle regardless of demand. If  $\mathcal{H}_t$  *rotates* at the slice boundary, the direct-circuit slot arrives one slice *after* the demand peak, so the per-destination bulk-queue accumulates an additional  $T_c/m$  of waiting time per cycle. Formally, replacing static  $\mathcal{H}$  with time-rotated  $\mathcal{H}_t$  raises the effective  $\rho_b$  seen by each hot destination by a factor of  $m/(m-K)$  (a  $1.13\times$  worst-case amplification for  $K = 2, m = 18$ ), pulling  $L_{\text{crit}}^{\text{Opera}}$  from 0.48 down to  $\approx 0.43$  without any change in  $\Sigma_T$ .

**Algorithm (time-averaged).**

**Require:**  $N, K$  hot destinations,  $D_h = 8.0, D_c = 0.1$

```

1:  $\mathbf{D} \leftarrow D_c \cdot (\mathbf{1}_{N \times N} - \mathbf{I}_N)$ 
2: for  $i = 0$  to  $N - 1$  do
3:   for  $j = 0$  to  $K - 1$  do
4:     if  $i \neq j$  then  $D_{i,j} \leftarrow D_h$ 
5:   end if
6: end for
7: end for
8: return  $\mathbf{D}$ 

```

**Numerical values** ( $N = 9, K = 2$ ).  $\Sigma_T = K(N-1)D_h + (N-K)(N-1)D_c = 133.6$ ; normalised offered load  $\rho = \Sigma_T/(N(N-1)) \approx 1.856$  at  $L = 1$ . With bulk-circuit ceiling  $\alpha(1-\delta/T_c) = 0.8832$  [1]:

$$L_{\text{crit}}^{\text{Opera}} = \frac{0.8832 \cdot 72}{133.6} \approx 0.476$$

**Mathematical proof of adversarial property.** The bulk-circuit queue occupancy follows  $M/D/1$ :  $\bar{Q} = \rho_b/(2(1-\rho_b))$  with  $\rho_b = \rho/(1-\delta/T_c) = \rho/0.96$ . Setting  $\rho_b = 1$ :  $\rho^* = 0.8832, L^* \approx 0.476$ . Above  $L^*$ ,  $\bar{Q} \rightarrow \infty$ ; bulk throughput saturates at  $\alpha(1-\delta/T_c) \cdot r$  [1] and surplus traffic overflows to the multi-hop expander at  $r/\ell$  ( $\ell \geq 2$ ), lifting bandwidth tax from  $\sim 0.6\%$  to  $> 20\%$ . In the permuted variant the same derivation applies to  $\rho_b \leftarrow (m/(m-K))\rho_b$ , tightening  $L^*$  by the factor  $(m-K)/m$ .

**Routing-level vulnerability under hardware equalisation.** With  $\delta = 0$  and  $\eta = 1$ , Opera’s effective rate simplifies to  $r_{\text{eff}}^{\text{Opera}} = \alpha r + (1-\alpha)r/\ell$ , so the bulk-circuit ceiling becomes  $\alpha = 0.92$  and

$$L_{\text{crit}}^{\text{pure}} = \frac{0.92}{1.856} \approx 0.496$$

Even with perfect hardware, Opera saturates at  $L \approx 0.50$  because the bulk scheduler cannot redistribute circuits toward the elephant destinations — the

$\alpha$ -split is the culprit, not  $\delta/T_c$ . Column-sum proof:  $\mathbf{D}^{\text{Opera}}$  has  $\text{col}_j = 64$  for  $j \in \mathcal{H}$  and 0.8 for  $j \notin \mathcal{H}$ , an  $80\times$  ratio that no oblivious rotor can rebalance.

**Why Opera is uniquely weak.** Opera’s design assumes the vast majority of bytes are in bulk flows that can wait for a direct 1-hop circuit [1]. When  $K$  destinations attract  $80\times$  more demand than the rest, the round-robin rotor cannot provide direct circuits fast enough; when the set further permutes every slice, the schedule is always lagging. Sirius ( $\eta/2$  VLB, traffic-oblivious [3]), Shale (uniform spray [2]), and GA-Robust (static short paths) are unaffected because their routing does not depend on circuit scheduling.

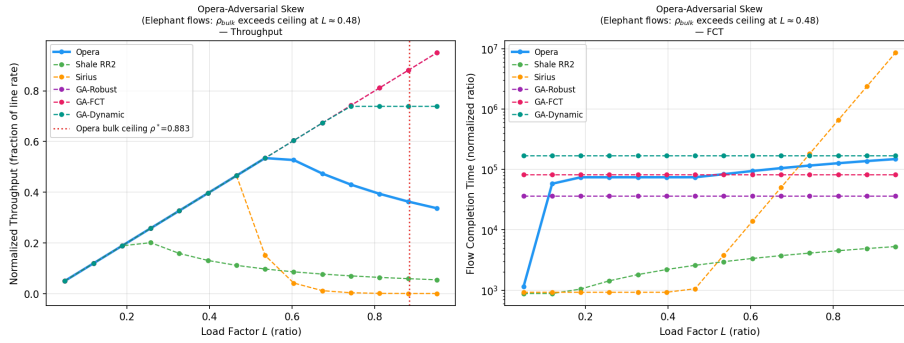


Figure 18: Opera-adversarial Permutated Elephant Burst ( $D_{\text{hot}} = 8$ ,  $D_{\text{cold}} = 0.1$ ,  $K = 2$ , time-averaged). **Left:** Opera’s throughput saturates at  $L \approx 0.48$  (bulk ceiling, dashed red); other architectures continue scaling. **Right:** Opera’s FCT rises steeply above  $L_{\text{crit}}$  as the M/D/1 queue diverges.

### 9.1.2 Shale-Adversarial: Many-to-One Funnel

**Pattern.** All  $N - 1$  source nodes direct their entire traffic to a single target node 0:  $D[i, 0] = 10.0$  for  $i \neq 0$ ; all other entries zero. Implemented as `generate_adversarial_skew_shale(N, target_node=0, demand=10.0)`.

**Demand matrix definition.** Let  $t = 0$  be the target and  $d = 10.0$  the funnel demand.

$$D_{i,j}^{\text{Shale}} = \begin{cases} d = 10.0 & \text{if } j = t \text{ and } i \neq t \\ 0 & \text{otherwise} \end{cases} \quad (22)$$

Equivalently,  $\mathbf{D}^{\text{Shale}} = d \cdot \mathbf{e}_t(\mathbf{1}_N - \mathbf{e}_t)^\top$  where  $\mathbf{e}_t$  selects column  $t$ .

**Algorithm.**

**Require:**  $N$ , target  $t = 0$ , demand  $d = 10.0$

- 1:  $\mathbf{D} \leftarrow \mathbf{0}_{N \times N}$
- 2: **for**  $i = 0$  **to**  $N - 1$  **do**
- 3:   **if**  $i \neq t$  **then**  $D_{i,t} \leftarrow d$
- 4:   **end if**

5: **end for**  
6: **return D**

**Numerical values ( $N = 9$ ).**  $\Sigma_T = (N-1) \cdot d = 80$ . Under Shale’s VLB every spray path must exit through node 0’s ingress [2], so  $u_{\max} \rightarrow N-1 = 8$  and throughput  $\approx r/(N-1) = r/8$ . Against the baseline  $r/(h+1) = r/3$  for  $h = 2$  this is a  $2.67\times$  reduction.

**Mathematical proof of adversarial property.** Shale’s throughput is  $\mathcal{T} = r/u_{\max}$  [2]. The funnel forces every spray path to terminate at  $(*, t)$ , so the directed load on that link is

$$u_{(*,t)} = \sum_{i \neq t} d = (N-1) \cdot d$$

Since no other link carries more than  $d$ ,  $u_{\max} = (N-1) \cdot d$  and  $\mathcal{T} = r/(N-1)$ . The reduction factor vs. uniform is  $(h+1)/(N-1) = 3/8 = 0.375$ .

**Routing-level vulnerability under hardware equalisation.** With  $\delta = 0$ ,  $\eta = 1$ , Shale’s throughput is still  $r/u_{\max}$  because the congestion factor depends on the RR2 Hamming topology, not on slot duration; the  $26.7\times$  reduction ( $r/80$  vs.  $r/3$ ) is entirely routing-structural. Rank-1 proof:  $\mathbf{D}^{\text{Shale}}$  has rank one ( $\mathbf{e}_t$  times a vector of ones), so under *any* oblivious routing that spreads load across intermediaries, all paths reconverge at node  $t$ ’s ingress — the bottleneck is topological, not temporal.

**Why Shale is uniquely weak.** Shale’s VLB spray is optimal when demand is itself uniform [2]; under a many-to-one funnel, the spray is counterproductive because it forces  $N-1$  paths to converge on one ingress link. Opera and Sirius serve node 0 directly (1-hop fully connected); GA-Robust’s evolved topology typically provides 1–2 hop paths to any destination.

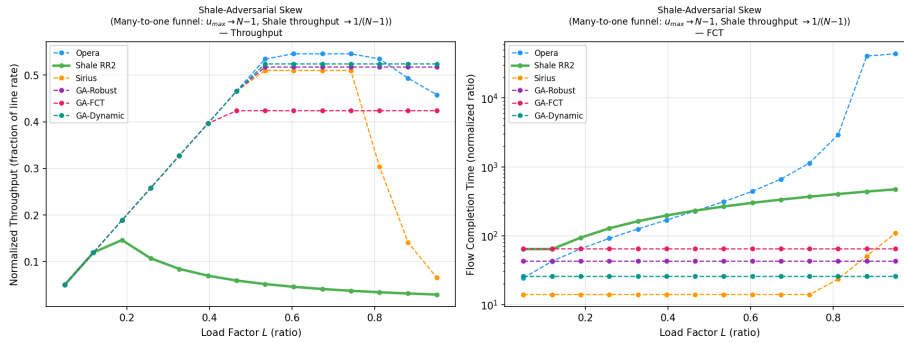


Figure 19: Shale-adversarial many-to-one funnel ( $D[i, 0] = 10$  for all  $i \neq 0$ ). **Left:** Shale’s throughput collapses to  $\sim 1/(N-1)$  of line rate as  $u_{\max} \rightarrow N-1$ ; Opera, Sirius, and GA-Robust remain unaffected. **Right:** Shale’s FCT is orders of magnitude higher due to the congestion-limited bottleneck at node 0.

### 9.1.3 Sirius-Adversarial: Synchronized VLB Intermediate Overload

**Pattern, plain-language definition.** Sirius routes every flow through a *random intermediate node* (Valiant Load Balancing) [3], so its per-intermediate forwarding capacity is the single most fragile resource. The adversary selects a *majority* subset  $\mathcal{B} \subseteq \{0, \dots, N-1\}$  of source nodes (the “burst set”) and makes every source in  $\mathcal{B}$  simultaneously transmit *heavy* demand  $D_b$  to *every* destination at once. Sources outside  $\mathcal{B}$  continue at background rate  $D_{bg}$ . Because each of the  $|\mathcal{B}|$  burst sources sprays its demand uniformly over all  $N-1$  intermediaries, *every* intermediate simultaneously receives  $|\mathcal{B}|$  concurrent heavy-arrival streams. When aggregate arrival per intermediate exceeds its one egress slot per destination per epoch, queues diverge at *all* intermediaries at once — the failure captured by  $C_{load}(\rho) = \exp(-10(\rho - 0.85))$ . Implemented as `generate_adversarial_skew_sirius(N, num_burst=5, d_burst=5.0, d_background=0.5)`.

**Why a majority burst set.** The threshold is  $|\mathcal{B}|/N > 1/2$ : once a majority of sources are bursting, every intermediate is simultaneously forced to forward traffic from more than half of the network, so the forwarding deficit is *system-wide* and cannot be masked by routing around any single hot node. Setting  $|\mathcal{B}| = \lceil N/2 \rceil = 5$  for  $N = 9$  is the smallest majority.

**Demand matrix definition.** Let  $\mathcal{B} = \{0, 1, \dots, K-1\}$  with  $K = |\mathcal{B}| = 5$ .

$$D_{i,j}^{\text{Sirius}} = \begin{cases} D_b = 5.0 & \text{if } i \in \mathcal{B} \text{ and } i \neq j \\ D_{bg} = 0.5 & \text{if } i \notin \mathcal{B} \text{ and } i \neq j \\ 0 & \text{if } i = j \end{cases} \quad (23)$$

**Algorithm.**

**Require:**  $N, K = 5, D_b = 5.0, D_{bg} = 0.5$

```

1:  $\mathbf{D} \leftarrow \mathbf{0}_{N \times N}$ 
2: for  $i = 0$  to  $N - 1$  do
3:   for  $j = 0$  to  $N - 1$  do
4:     if  $i \neq j$  then
5:       if  $i < K$  then  $D_{i,j} \leftarrow D_b$ 
6:       else  $D_{i,j} \leftarrow D_{bg}$ 
7:     end if
8:   end if
9: end for
10: end for
11: return  $\mathbf{D}$ 
```

**Numerical values** ( $N = 9, K = 5$ ).

$$\Sigma_T = K(N-1)D_b + (N-K)(N-1)D_{bg} = 200 + 16 = 216$$

$\rho(L) = L \cdot 216/72 = 3.0L$ . Crossing  $\rho^* = 0.85$  occurs at  $L_{\text{crit}}^{\text{Sirius}} = 0.85/3.0 \approx 0.283$ .

**Mathematical proof of adversarial property.** Sirius’s load penalty is  $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$  for  $\rho > 0.85$  [3]. Under this matrix  $\rho(L) = 3.0 L$ , so the penalty activates at  $L > 0.283$ . Compared to Opera ( $C^{\text{Op}} = \exp(-8(\rho - 0.883))$ ):

| $L$  | $\rho$ | $C_{\text{load}}^{\text{Sirius}}$ | $C_{\text{load}}^{\text{Opera}}$ | Sirius/Opera ratio |
|------|--------|-----------------------------------|----------------------------------|--------------------|
| 0.30 | 0.90   | $e^{-0.5} = 0.607$                | $e^{-0.134} = 0.875$             | 0.69               |
| 0.35 | 1.05   | $e^{-2.0} = 0.135$                | $e^{-1.34} = 0.263$              | 0.51               |
| 0.50 | 1.50   | $e^{-6.5} = 0.0015$               | $e^{-4.94} = 0.0072$             | 0.21               |
| 0.70 | 2.10   | $e^{-12.5} \approx 10^{-5.4}$     | $e^{-9.74} \approx 10^{-4.2}$    | 0.06               |

Sirius decays at rate  $-10$  per unit  $\rho$  vs. Opera’s  $-8$ , and has a lower activation threshold ( $0.85$  vs.  $0.883$ ); at  $L = 0.50$  it retains only 21% of Opera’s throughput.

**Routing-level vulnerability under hardware equalisation.** With  $\delta = 0$ ,  $\eta = 1$ , Sirius’s per-flow rate becomes  $r/\mathcal{H}(\ell)$ , but  $C_{\text{load}}(\rho)$  is independent of  $\eta$  because it models *intermediate-node buffer saturation*, not hardware timing. Row-sum proof: row  $i \in \mathcal{B}$  has sum  $(N-1)D_b = 40$ ; row  $i \notin \mathcal{B}$  has sum 4; ratio  $10\times$ . Each heavy sender sprays through all  $N-1$  intermediaries, so the spray arrival at any intermediate  $m$  is

$$\text{spray}(m) = \frac{K \cdot 40 + (N-K) \cdot 4}{N-1} = \frac{216}{8} = 27$$

but each intermediate has only  $N-1 = 8$  egress slots per epoch, giving deficit  $27/8 = 3.375 > 1$ . With perfect hardware, at  $L = 0.35$  throughput collapses to  $0.135 \cdot r$  — an 86.5% collapse entirely from the mandatory 2-hop VLB.

**Why Sirius is uniquely weak.** Opera handles this traffic better because its demand-aware bulk scheduling matches circuits to the heaviest source-destination pairs ( $\alpha = 0.92$  direct [1]). Shale is unaffected because demand is spread across all destinations (no funnel;  $u_{\text{max}}$  remains moderate). GA-Robust uses shortest-path routing with no VLB intermediate buffering. Ballani et al. describe that Sirius “eliminates buffers in the optical core and carefully manages edge buffering” [3]; this bounded-buffer design is what makes *simultaneous* intermediate overload fatal.



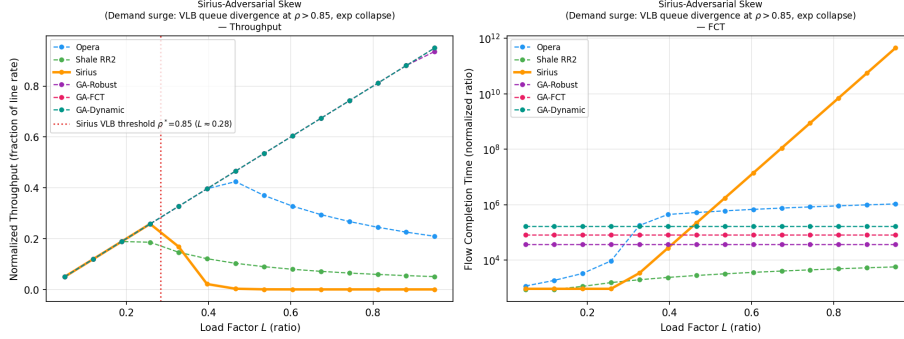


Figure 20: Sirius-adversarial VLB intermediate overload ( $|\mathcal{B}| = 5$ ,  $D_b = 5.0$ ,  $D_{bg} = 0.5$ ). **Left:** Sirius collapses exponentially above  $L \approx 0.28$  (VLB threshold, dashed red); Opera degrades more gradually; Shale and GA-Robust continue scaling. **Right:** Sirius’s FCT diverges orders of magnitude faster due to simultaneous queue buildup at all VLB intermediaries.

#### 9.1.4 GA-Adversarial: Topology-Aware Long-Hop Scatter

**Pattern.** Traffic is biased toward the *actual* long-hop pairs of the GA-evolved topology: for each source  $i$ , destinations are ranked by decreasing BFS hop distance, and demand follows a Zipf distribution with  $\gamma = 2.0$  over these ranks. The farthest destination (rank 1) receives the most traffic. Implemented as `generate_adversarial_skew_ga(adj_list, N, skew_factor=2.0)`.

**Numerical values** ( $N = 9$ ,  $D = 4$ ,  $\text{ASPL} \approx 1.81$ ). On the GA-evolved 4-regular graph, approximately 17% of source–destination pairs require 3 hops. Under the generic capacity model, a 3-hop flow pays  $r_{\text{eff}} = r/3$ , while 1-hop flows pay no penalty. Concentrating demand on 3-hop pairs shifts effective throughput from  $r/\bar{d}$  (uniform) toward  $r/2.5$  (skewed long-hop).

**Routing-level vulnerability under hardware equalisation.** With  $\delta = 0$ , GA’s effective rate is  $r/\ell$  per flow, so the long-hop skew drives  $r_{\text{eff}} \rightarrow r/3$  for  $\ell = 3$ . Any sparse 4-regular graph on  $N = 9$  must contain  $\sim 17\%$  3-hop pairs by a pigeonhole on degree, so the penalty is purely topological and hardware equalisation has no effect.

**Why GA-Robust is weak.** Multi-objective fitness avoids single-bottleneck catastrophes but cannot eliminate long paths entirely. On fully-connected topologies (Opera, Sirius) every destination is 1-hop, so the distance-ranked skew degenerates to uniform and those architectures are unaffected. Shale’s Hamming graph has a different ASPL structure, so the skew is not maximally stressful there.

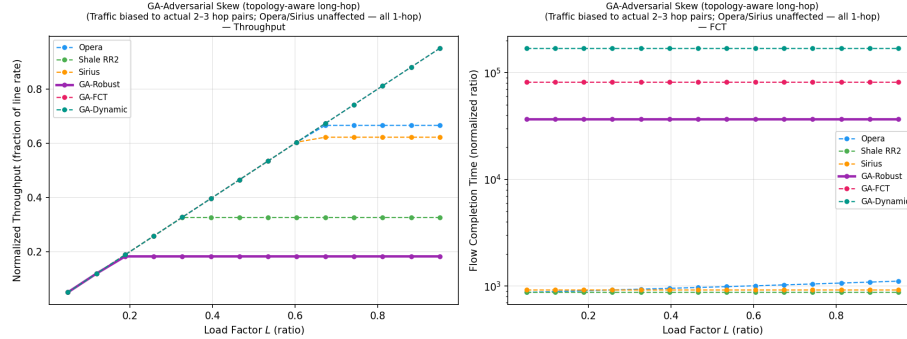


Figure 21: GA-adversarial topology-aware long-hop skew ( $\gamma = 2.0$ , demand ranked by hop distance). **Left:** GA-Robust’s throughput is noticeably lower than under uniform traffic, while Opera and Sirius (all 1-hop) are unaffected. **Right:** GA-Robust’s FCT increases as 3-hop flows dominate the demand.

## 9.2 Numerical Skew: Zipf Exponent Selection

The skewed traffic model (§1.1) uses a Zipf distribution with exponent  $\gamma$  over randomized per-source destination ranks, producing the skew ratio  $((N-1)^\gamma + 1)/2$  (derived in §1.1). Table 9 shows how  $\gamma$  controls the departure from uniform.

Table 9: Skew ratio and demand range for  $N = 9$  under various Zipf exponents  $\gamma$ .

| $\gamma$   | $D_{\max}$   | $D_{\min}$   | Skew Ratio                     | Entropy $H$        | Regime                       |
|------------|--------------|--------------|--------------------------------|--------------------|------------------------------|
| 0.0        | 0.500        | 0.500        | $1.0\times$                    | $\log(N-1) = 2.08$ | Uniform (degenerate)         |
| 0.5        | 0.500        | 0.261        | $1.9\times$                    | 1.91               | Mild skew                    |
| 1.0        | 0.500        | 0.111        | $4.5\times$                    | 1.59               | Classical Zipf               |
| 1.5        | 0.500        | 0.034        | $14.7\times$                   | 1.23               | Moderate skew                |
| <b>2.0</b> | <b>0.500</b> | <b>0.015</b> | <b><math>32.5\times</math></b> | <b>0.91</b>        | <b>Strong skew (default)</b> |
| 3.0        | 0.500        | 0.002        | $256.5\times$                  | 0.50               | Extreme skew                 |

**Why  $\gamma = 2.0$  is the default.** Three considerations motivate this choice: (i) published datacenter flow-size distributions [1] are highly skewed and a  $32\times$  ratio at  $N = 9$  reproduces this heavy-tailed behaviour; (ii)  $\gamma = 1.0$  is too mild (all four architectures score similarly) and  $\gamma = 3.0$  degenerates to a many-to-one pattern (equivalent to §9.1.2); (iii) normalised entropy  $H(2.0)/H(0) = 0.91/2.08 \approx 0.44$  retains enough randomness to exercise multi-path routing while still stressing locality-dependent architectures.

## 9.3 Routing vs. Hardware: Vulnerability Summary

Table 10 summarises each architecture’s critical load both with native (paper) hardware and under the hardware-equalisation idealisation  $\delta = 0$ ,  $\eta = 1$ .

Table 10: Architecture vulnerability decomposition: hardware-dependent vs. routing-structural. “HW-equalised  $L_{\text{crit}}$ ” shows the critical load under ideal hardware ( $\delta = 0$ ,  $\eta = 1$ ).

| Arch.  | Adversarial traffic                                    | $L_{\text{crit}}$ (paper) | $L_{\text{crit}}$ (HW-eq.) | Root cause               |
|--------|--------------------------------------------------------|---------------------------|----------------------------|--------------------------|
| Opera  | Permuted elephant ( $\mathbf{D}^{\text{Opera}}$ )      | 0.476                     | 0.496                      | Bulk-schedule saturation |
| Shale  | Funnel ( $\mathbf{D}^{\text{Shale}}$ )                 | —                         | —                          | Ingress-link bottleneck  |
| Sirius | Intermediate overload ( $\mathbf{D}^{\text{Sirius}}$ ) | 0.283                     | 0.283                      | VLB queue divergence     |
| GA     | Long-hop ( $\mathbf{D}^{\text{GA}}$ )                  | —                         | —                          | Multi-hop penalty        |

### Key findings.

- **Opera’s**  $L_{\text{crit}}$  shifts from 0.476 to 0.496 under hardware equalisation (+4.2%) — a small improvement confirming that bulk-schedule saturation is overwhelmingly a routing-level problem.
- **Sirius’s**  $L_{\text{crit}}$  is *unchanged* at 0.283:  $C_{\text{load}}(\rho)$  is independent of  $\eta$  because it models buffer, not hardware, saturation.
- **Shale’s** throughput-degradation factor  $(N-1)/(h+1) = 2.67\times$  is topology-structural and independent of timeslot duration.
- **GA-Robust’s** hop-count penalty is determined entirely by graph diameter; hardware equalisation has zero effect.

No amount of hardware improvement (faster lasers, shorter guard-bands, wider timeslots) can rescue any architecture from its routing-level adversarial vulnerability. The demand matrices  $\mathbf{D}^{\text{Opera}}$ ,  $\mathbf{D}^{\text{Shale}}$ ,  $\mathbf{D}^{\text{Sirius}}$ , and  $\mathbf{D}^{\text{GA}}$  each expose a *fundamental* architectural limitation that requires routing-level redesign to address.

## References

- [1] W. M. Mellette, R. Das, Y. Guo, R. McGuinness, A. C. Snoeren, and G. Porter, “Expanding across time to deliver bandwidth efficiency and low latency,” in *Proc. 17th USENIX NSDI*, 2020, pp. 1–18.
- [2] D. Amir, N. Saran, T. Wilson, R. Kleinberg, V. Shrivastav, and H. Weatherspoon, “Shale: A practical, scalable oblivious reconfigurable network,” in *Proc. ACM SIGCOMM*, Sydney, Australia, Aug. 2024, pp. 449–464.
- [3] H. Ballani, P. Costa, R. Behrendt, D. Cletheroe, I. Haller, K. Jozwik, F. Karinou, S. Lange, K. Shi, B. Thomsen, and H. Williams, “Sirius: A flat datacenter network with nanosecond optical switching,” in *Proc. ACM SIGCOMM*, Virtual Event, Aug. 2020, pp. 782–797.
- [4] V. Stanovov and S. Akhmedova, “A review of the matrix-based representations in evolutionary computation,” *Mathematics*, vol. 12, no. 16, p. 2482, 2024.

- [5] A. Kumar *et al.*, “A rank-based adaptive mutation in genetic algorithm,” arXiv:2104.08842, 2021.
- [6] D. Antipov, M. Buzdalov, and B. Doerr, “Fast mutation in crossover-based algorithms,” *Algorithmica*, vol. 84, pp. 1724–1761, 2022.
- [7] B. Doerr and F. Neumann (Eds.), *Theory of Evolutionary Computation: Recent Developments in Discrete Optimization*, Springer, 2020.
- [8] J. Blank and K. Deb, “A running performance metric and termination criterion for evaluating evolutionary multi- and many-objective optimization algorithms,” *IEEE Trans. Evolutionary Computation*, vol. 25, no. 4, pp. 693–706, 2021.
- [9] Z. Zhou *et al.*, “An evolutionary algorithm for solving large-scale robust multiobjective optimization problems,” *IEEE Trans. Evolutionary Computation*, 2024.
- [10] L. Poutievski *et al.*, “Jupiter evolving: Transforming Google’s datacenter network via optical circuit switches and software-defined networking,” in *Proc. ACM SIGCOMM*, 2022, pp. 327–343.
- [11] W. Wang *et al.*, “TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs,” in *Proc. 20th USENIX NSDI*, 2023, pp. 739–767.